DAT560 Project Report

Evaluating Multimodal Retrieval-Augmented Generation on MMDocIR

DAT560 Project Team 6 (https://github.com/dat560-2026/project-team-6)
University of Stavanger, Norway
k.vagen@stud.uis.no,oltog2574@uis.no,or.ostlyngen@stud.uis.no,gp.israel@stud.uis.no

ABSTRACT

This project investigates multimodal retrieval-augmented generation (RAG) through a structured comparison of three system designs: a baseline retrieval pipeline (System 1), an optimized multimodal pipeline (System 2), and an agentic pipeline with dynamic decision-making (System 3). The systems are evaluated on a document-based question answering task using both retrieval and generation metrics.

System 2 achieves the strongest overall performance, improving Precision@1 from 0.5933 to 0.8667 and Token F1 from 0.1837 to 0.2658 compared to the baseline. These gains are primarily driven by improvements in chunking strategy, multimodal retrieval, and prompting. In particular, fixed-size chunking (2k) and multimodal retrieval significantly enhance context preservation and answer quality, while Chain-of-Thought prompting improves performance on complex queries.

In contrast, the agentic system does not outperform the baseline despite substantially higher computational cost, highlighting the challenges of multi-step decision pipelines under limited model capacity. The results show that retrieval quality is the dominant factor in end-to-end performance, and that carefully optimized non-agentic pipelines can outperform more complex architectures.

Overall, the findings emphasize the importance of balancing retrieval effectiveness, generation quality, and computational efficiency in the design of multimodal RAG systems.

KEYWORDS

retrieval-augmented generation, multimodal RAG, MMDocIR, Qdrant, query transformation, chunking, prompting, agentic systems

1 INTRODUCTION

Retrieval-Augmented Generation (RAG) is one of the most practical ways to improve large language model (LLM) performance on knowledge-intensive tasks. Instead of relying exclusively on the model's internal parameters, a RAG system retrieves relevant external evidence and provides that material as context during answer generation. This reduces hallucinations, increases grounding, and makes it possible to update the knowledge source without retraining the underlying model.

Our project investigates this design space in a multimodal setting. We focus on the MMDocIR benchmark, where questions may depend on information distributed across long PDF documents and, in more advanced settings, across multiple modalities such as text, tables, images, and page layout structure. This creates a realistic challenge for modern document QA systems because the system must not only retrieve relevant content, but retrieve the *right representation* of the content before answer generation.

Research Problem. The central research question in this project is: *which combinations of preprocessing, retrieval, prompting, and control-flow strategies improve answer quality in a multimodal RAG pipeline, while still keeping runtime and implementation complexity manageable?* More concretely, the project addresses the following technical problems:

(1) how to preprocess PDF documents into chunks that preserve enough context for downstream retrieval;
(2) how to compare alternative chunking strategies and embedding models;
(3) how to improve retrieval through query transformation and multimodal evidence handling;
(4) how to structure prompting so the generator answers directly from retrieved context; and
(5) how to determine whether a more agentic, LLM-driven control flow can outperform a strong non-agentic baseline.

Project Structure. To answer these questions, we implemented three systems of increasing complexity. System 1 is a baseline RAG pipeline with preprocessed chunks, hybrid retrieval, and constrained answer generation. System 2 extends the baseline with several query-processing methods, multiple chunking strategies, multimodal retrieval support, and alternative prompting strategies. System 3 explores a more agentic setup where decisions for query technique, context relevance and prompting strategy for a given question is made by the LLM.

Main Contributions. The report is organized around the following contributions:

(1) a modular codebase for preprocessing, indexing, retrieval, and answer generation;
(2) an implementation of several chunking, query transformation, and prompting strategies within one comparable evaluation framework;
(3) a baseline-to-agentic progression that enables direct comparison between simpler and more adaptive system designs; and
(4) a reproducible evaluation workflow that measures both quality and efficiency.

Notes on AI Usage. Generative AI tools were used as coding and writing assistants during the project, primarily for iterative drafting, refactoring support, and language polishing. Final implementation choices, experiments, and report structure were still curated manually within the project repository.

Supervised by DAT560 course staff.

Project in Generative AI (DAT560), UiS, Norway
2026.

2 BACKGROUND (AND RELATED WORK)

2.1 RAG Fundamentals

Retrieval-Augmented Generation (RAG) is a framework first introduced by Lewis et. al. in their "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" paper. RAG provides a way to improve language generation in knowledge-intensive tasks. Particularly, RAG is used to enhance LLM capabilities by addressing their inability to easily update and retrieve up-to-date knowledge and information, alongside their tendency to produce "hallucinations". RAG achieves this by combining pre-trained parametric memory with an external non-parametric memory (typically a dense vector index of some knowledge source), and utilizes a neural retriever to identify the top-K relevant documents for a given query, which are then provided as additional context to the generator. RAG achieves these things, without having to retrain the entire model [1].

2.2 MMDocIR Dataset

MMDocIR is a large-scale benchmark for multi-modal document retrieval. The dataset consists of 313 long documents in the evaluation set and 6,878 documents in the training set, which totals over 75,000 question–answer pairs. The dataset spans diverse domains and modalities including text, images, tables, and layout elements, and supports both page-level and fine-grained layout-level retrieval tasks. Each question is annotated with evidence at both the page level and specific layout regions (e.g., tables, figures, equations), which allows for a detailed evaluation of retrieval performance. With its multi-page, cross-modal, and reasoning-heavy queries, MMDocIR offers a realistic and demanding benchmark for assessing the capabilities of advanced RAG systems [10].

2.3 Embedding and Retrieval Backends

At the core of modern retrieval systems are *embedding vectors*: fixed-length numerical representations of text or images produced by neural encoders. These embeddings map semantically similar inputs to nearby points in a shared latent space. At query time, inputs are encoded into the same space and relevant documents are retrieved using similarity measures such as cosine similarity or dot product, enabling meaning-based search rather than exact keyword matching.

These vectors are typically stored in a *vector database* (e.g., *Qdrant* [4]), which supports efficient, high-performance search via approximate nearest neighbour (ANN) indexing. This allows top-k retrieval in sub-linear time, while also enabling metadata-based filtering to constrain results based on document attributes.

2.3.1 ***Dense Retrieval.*** Dense retrieval represents both queries and documents as embedding vectors in a continuous vector space. Relevance is determined by geometric proximity, enabling the system to capture semantic relationships beyond surface-level text. This makes dense retrieval particularly effective for handling paraphrased queries and capturing contextual meaning.

2.3.2 ***Sparse Retrieval.*** Sparse retrieval methods rely on lexical matching between queries and documents. A widely used approach is *BM25* [8], which scores documents based on term frequency and inverse document frequency. Unlike dense retrieval, sparse methods excel when queries contain precise terms such as identifiers, acronyms, or rare keywords, where exact matching is critical.

2.3.3 ***Hybrid Retrieval.*** Dense and sparse retrieval approaches have complementary strengths, and modern systems often combine them in a hybrid framework. One common technique is *Reciprocal Rank Fusion* (RRF) [9], which merges multiple ranked lists by aggregating their rank positions rather than raw scores. This avoids issues arising from differing scoring scales and emphasises documents that consistently rank highly across methods.

Overall, hybrid retrieval improves robustness by incorporating both semantic similarity and exact term matching, reducing the likelihood that any single retrieval strategy misses relevant documents.

2.4 Query Transformation Techniques

Query transformation is the process of optimizing, expanding, or restructuring user queries to improve retrieval accuracy. This is important because user queries are often suboptimal. They may be too broad or too narrow, or use words that differ from those used in relevant documents. In the following, we describe some common query transformation techniques used to improve retrieval performance [11].

2.4.1 ***HyDE.*** Hypothetical Document Embeddings (HyDE) is a technique in which, instead of directly using the query for retrieval, a large language model (LLM) generates a hypothetical answer to the query. The embedding of this generated answer is then used to retrieve relevant documents. This approach is particularly useful when there is a mismatch between the wording of the query and the documents, as the generated answer can be semantically closer to the relevant content, even if it is not factually correct.

2.4.2 ***Multi-Query.*** Multi-Query Generation is a technique in which, instead of relying on a single user query that may miss relevant documents due to differences in wording or phrasing, a large language model generates multiple paraphrases of the original query. Each of these queries is then embedded and used to retrieve the top-k documents. The retrieved documents are subsequently combined and reranked using a chosen method, after which the final top-k documents are selected.

2.4.3 ***RAG-Fusion.*** RAG-Fusion is a technique that extends multi-query generation by combining retrieval results using Reciprocal Rank Fusion (RRF) instead of a standard reranking method [12]. In this approach, multiple query variations are generated using a large language model, and each query is used to retrieve a set of documents independently. The final ranking is then produced by aggregating the individual rankings using RRF, which assigns higher scores to documents that consistently appear across multiple query results. This improves robustness by favoring documents that are repeatedly retrieved across different query formulations.

2.4.4 ***Step-Back Prompting.*** Step-Back Prompting is a prompting technique introduced by Huaixiu et al. (2023) in "Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models" [13]. It involves using a large language model to abstract an original question into a more general version. For example, the question "Who was the first Black president of the United States?"

can be reformulated into a broader question such as "Who are the presidents of the United States?". While this technique is primarily designed to improve reasoning in LLMs, it can also be adapted for retrieval tasks. In this setting, the abstracted query is used alongside the original query to retrieve relevant documents. The combined results are then reranked, and the final top-k documents are selected. This allows the system to capture both specific and more general context, which can improve retrieval performance.

2.4.5 ***Query Decomposition.*** Query Decomposition is a technique in which a complex query is broken down into smaller subquestions using a LLM [14]. This allows the system to retrieve relevant documents not only for the original query as a whole, but also for its individual components. Each sub-question is used to perform retrieval independently, and the resulting documents are then combined and reranked. The final top-k documents are selected based on this aggregated ranking. This approach can improve retrieval performance for complex queries that involve multiple aspects or require multi-step reasoning.

2.4.6 ***Query Rewriting.*** Query Rewriting is a technique in which a query is reformulated by a large language model to improve its semantic clarity and effectiveness for retrieval [15]. The goal is to produce a rewritten query that better captures the user's intent and is more aligned with relevant documents in the collection. This can improve retrieval performance by reducing ambiguity and increasing the likelihood of retrieving relevant documents that do not contain unnecessary or unrelated information.

2.4.7 ***Query Expansion.*** Query Expansion is a technique in which a user query is improved by a large language model by adding relevant contextual information such as synonyms, alternative phrasing, related terms, and broader concepts [16]. This helps address cases where queries are ambiguous or where the vocabulary used does not match the terminology in relevant documents. For example, the query "car maintenance" may be expanded to include terms such as "vehicle servicing", which may appear in relevant documents.

2.5 Multimodal and Agentic RAG

Multimodal RAG extends the classical text-only setting by incorporating evidence from sources such as images, tables, and page layouts. This matters because many real documents encode important meaning outside plain paragraphs. Agentic RAG pushes adaptation one step further by letting an LLM decide which tools or sub-strategies to use during the processing workflow.
LangChain is an open-source orchestration framework for application development using LLMs, providing a modular, abstraction-based library that simplifies building LLM-driven applications. Within the LangChain ecosystem, *LangGraph* is a specialized agent framework that "uses the power of graph-based architectures to model and manage the intricate relationships between various components of an AI agent workflow." LangGraph enables developers to build agentic workflows where different agents interact and collaborate. Unlike traditional RAG applications where the LLM retrieves and directly responds using a vector database, agentic RAG applications can orchestrate multiple decision-making agents that autonomously determine the next steps, before producing a final response [21].

3 METHODOLOGY, DESIGN & IMPLEMENTATION

This section describes how the DAT560 project is implemented in the repository, from PDF preprocessing to answer generation and evaluation.

3.1 Dataset & Split Policy

Before further code development, we inspected the dataset to understand its structure, answer types, and document coverage, which provided useful insight into the components of our preprocessing pipeline. The MMDocIR subset used in this project contains 750 queries, split into 600 training and 150 test queries. It includes 131 PDF documents overall. Training queries reference all 131 documents, while test queries reference 95 documents, all of which are also present in the training set. This indicates a query-level split over a shared document corpus rather than a disjoint document-level partition. The training split contains both pure-text and image-oriented questions. We adhered to the provided split as-is, and no fine-tuning or test-set adaptation was performed on the embedder, retriever, or generator. The test split was used only for final evaluation of Systems 1, 2, and 3.

3.2 Data Extraction & Preprocessing

For preprocessing, we extracted text from MMDocIR PDFs using the `docling` library with fast-mode partition extraction (strategy="fast"), which preserves block-level structure (titles, headings, and narrative text) together with page numbers while allowing header/footer filtering. For optimization purposes, we implemented parallel PDF processing using Python's `ProcessPoolExecutor` with 7 CPU workers, which significantly reduced extraction time.

3.2.1 ***Fixed-size Chunking.*** Concatenates extracted blocks sequentially until some specified length, then starts a new chunk. This provides uniform chunk sizes, but may split logical content boundaries.

3.2.2 ***Sliding-window Chunking.*** Uses fixed-size windows with a 200-character overlap between consecutive chunks. The idea of this overlap is to preserve context at chunk boundaries, which should improve retrieval coherence for dense, continuous text documents.

3.2.3 ***Semantic Chunking.*** Groups blocks by natural document structure; particularly, each heading or title represents the start of a new chunk, and body text then accumulates until the next heading. This approach preserves the logical semantic boundaries, but may produce variable-sized chunks. Also, consecutive headers without any body text are merged in order to prevent any orphaned content.

3.2.4 ***Hierarchical Chunking.*** Creates a two-level hierarchy within each page; specifically, a parent chunk contains the full page, and child chunks represent section-level content separated by headings. This approach enables coarse-to-fine retrieval, matching on page-level content first and then retrieving fine-grained sections.

3.2.5 ***Enhanced Hierarchical Chunking.*** Enhanced hierarchical chunking is an optimized variant that operates across page boundaries. Sections spanning multiple pages are kept together as single logical units, reducing context fragmentation. Each section maintains paragraph-level children, and titles are prepended to child chunks to preserve context.

3.2.6 Notes on Metadata and Configuration. All these strategies preserve any page number metadata for the retrieval evaluation. The strategy to be used can be configured at runtime, enabling ablation studies to assess how these various strategies influence the retrieval performance.

3.3 Embedding

Embeddings are produced using `Jina CLIP v2` [5], a multimodal model that maps both text and images into a shared 1024-dimensional vector space. Encoding is performed via the `SentenceTransformer` interface, with all output vectors L2-normalised so that cosine similarity is equivalent to dot product. This enables direct comparison between text and image representations within a unified embedding space.

`Jina CLIP v2` supports prompt-aware encoding, which is used to distinguish indexing from querying. Document chunks are encoded with `prompt_name="document"`, while queries use `prompt_name="retrieval.query"`. This asymmetric encoding aligns with the model's contrastive training objective, where query and document representations are optimised for different roles. Batching is used during indexing to manage memory constraints, with different batch sizes for text and image data reflecting their respective computational costs.

3.4 Indexing

All chunk types (text, page images, figures, and evidence blocks) are stored in a single Qdrant collection [4] within the shared 1024-dimensional cosine-distance space produced by the embedder. This unified indexing strategy allows a single similarity query to span all modalities, while type-based filtering at query time controls which chunk types are eligible for retrieval.

Collection management follows a reuse-first policy: at startup, the pipeline checks whether a collection already exists and contains indexed data; if so, the existing index is reused without rebuilding. A `force_rebuild` flag is provided to trigger full reconstruction when necessary, such as after changes to the chunking strategy. Each stored chunk includes a metadata payload containing its type, source document, page numbers, and identifier, supporting both fine-grained retrieval filtering and downstream evaluation.

3.5 Retrieval

The base retrieval component is a `HybridRetriever` that combines dense vector search with `BM25` sparse retrieval [8] using *Reciprocal Rank Fusion* (RRF) [9]. At query time, both rankers independently retrieve a candidate pool larger than the requested top-k to improve ranking robustness. The resulting ranked lists are then merged by computing an RRF score for each document:

$$s_{\text{RRF}}(d) = \frac{1}{60 + r_{\text{dense}}(d)} + \frac{1}{60 + r_{\text{BM25}}(d)}$$

where $r(d)$ denotes the zero-based rank of document d in each list. The constant 60 smooths contributions, preventing highly ranked documents from disproportionately dominating the final ranking.

`BM25` complements dense retrieval by capturing exact lexical matches, making it particularly effective for queries involving identifiers, codes, or rare proper nouns that may not be well represented in semantic space.

To prevent visual chunks from occupying candidate slots during text-focused retrieval, type-based filtering is applied to both the dense and sparse retrieval stages. By default, only text chunks are considered in hybrid retrieval, while visual evidence is retrieved separately through a dedicated multimodal routing mechanism (Section 3.7).

All query transformation techniques operate as wrappers around this base retriever. Standard retrieval invokes the hybrid retriever directly, while transformation-based methods generate one or more modified queries, retrieve independently for each, and merge the results by deduplicating on chunk identifiers and retaining the highest-scoring occurrence of each chunk.

3.6 System 1 (Baseline RAG)

3.6.1 Architecture and Pipeline Overview. The baseline architecture combines preprocessed chunks with indexing, retrieval, and generation. Document chunks are embedded using `Jina CLIP v2` (1024D), and these embeddings are indexed in Qdrant using cosine distance for efficient similarity search. Retrieval uses a hybrid approach that combines `BM25` (exact lexical matching) and dense vector retrieval via Qdrant (semantic matching). Results from both methods are merged using RRF ($k = 60$), balancing sparse and dense signals. The top-$k = 5$ documents are then passed to generation. The generator uses Ollama `qwen3-vl:8b-instruct`, with a system prompt that constrains answers to the provided context. Retrieved documents are concatenated as context and provided alongside the user query. Finally, retrieval and generation performance are evaluated using the metrics defined in this section.q

3.6.2 Prompt Template. We curated a grounding-focused system prompt to minimize hallucinations and ensure that answers are derived solely from retrieved context. The prompt explicitly instructs the model to: (1) read the entire context before responding, (2) provide only direct answers without preamble or explanation, (3) handle special cases (yes/no questions, multiple answers, list formatting), and (4) perform simple calculations (e.g., counting and ratios) when answers can be derived from context. Furthermore, temperature is set to 0.0 and top_p to 0.1, enforcing deterministic, low-variance outputs.

3.6.3 Evaluation of Text-Only Baseline on Full Multimodal Test Set. System 1 is a baseline text-only system that uses text embeddings from `Jina CLIP v2` and does not process images or handle table-based evidence. However, to ensure fair comparison across all three systems, System 1 is evaluated on the complete test set, which includes diverse question types (pure-text questions, chart-based questions, figure-based questions, table-based questions, and layout-based questions). Although System 1 is not designed to handle multimodal evidence, evaluating it on the full test distribution provides a realistic baseline performance and allows us to measure

the actual performance gap that Systems 2 and 3 (both of which incorporate multimodal retrieval) must overcome. This is to ensure a direct and fair comparison of how multimodal approaches improve over the baseline.

3.7 System 2 (Advanced mRAG)

3.7.1 Overall Architecture. System 2 extends the baseline along three orthogonal dimensions, all operating over the same Qdrant index: (1) query transformation techniques that rewrite or expand the user query prior to retrieval; (2) a multimodal retrieval layer that detects visually oriented questions and retrieves image evidence alongside text; and (3) a set of prompting strategies that control how the vision-language model (VLM) interprets and formats responses based on retrieved context. Each dimension is independently configurable, enabling controlled ablation across query processing, chunking strategy, and prompting design.

3.7.2 Multimodal Retrieval. The system extends traditional text-based retrieval by incorporating visual information through a vision-language model. Queries are first classified to determine whether visual evidence is required. For such queries, both textual and image-based context are retrieved and jointly used during answer generation, enabling the system to handle information presented in figures, diagrams, and layouts that are not captured in text alone.

3.7.3 Query Transformation Techniques. Query transformation in System 2 is implemented as a modular layer on top of the base `HybridRetriever`, following the techniques outlined in Section 2.4. Each method wraps the retriever and exposes a unified `retrieve(question, top_k)` interface, ensuring seamless integration with the existing pipeline. Refer to Section 3.5 for more details on retrieval. This design enables all techniques to be evaluated under identical retrieval conditions, isolating the effect of query reformulation.

3.7.4 Preprocessing. Prior to chunking, documents are preprocessed to remove noise and improve text quality for embedding. Initial experiments with the unstructured library introduced artifacts such as headers, footers, and repeated layout elements, which negatively affected retrieval performance.

To address this, we adopt docling, which enables structured, block-level parsing and filtering of non-content elements. Additional normalization steps, including merging fragmented headings and removing duplicated content, are applied to ensure more coherent chunk boundaries.

3.7.5 Chunking Strategy Comparison. System 2 evaluates the five chunking strategies described in Section 3.2 as an independent axis of analysis. Each strategy produces a distinct `prompt_preprocessed_chunks.json` file, and switching strategies requires only a configuration change followed by a forced index rebuild. This separation ensures that the impact of chunk granularity and boundary design can be evaluated under consistent retrieval and generation conditions.

3.7.6 Multimodal Retrieval. The multimodal retrieval layer extends any query technique with a two-stage visual routing mechanism, implemented via the `MultimodalRetriever`.

Visual classification. Each incoming query is first classified to determine whether visual evidence (e.g. figures, charts, tables, or diagrams) is required. This is performed using an LLM prompted with a structured few-shot schema, returning a JSON object of the form `{"visual": true/false}`. When this information is available from dataset metadata, the LLM call is skipped.

Retrieval budget allocation. The top-k retrieval budget is partitioned between text and image evidence. Non-visual queries allocate the full budget to text (ratio = 1.0), while visual queries reserve a fixed portion (20%) for image retrieval (ratio = 0.8). This prevents text-only dominance in queries requiring visual grounding.

Document-filtered image retrieval. When an image budget is allocated, the query is encoded using the `Jina CLIP v2` image encoder and used to search over image-related chunk types (`page_image`, `figure`, `evidence`) in Qdrant. A document-level filter restricts the search to documents already retrieved by the text pipeline, reducing the risk of retrieving contextually irrelevant images.

Score-preserving fusion. Text and image retrieval operate on different scoring scales (RRF vs. cosine similarity). To ensure consistent ranking, each text result retains its raw dense similarity score in addition to its RRF score. Image results are then inserted into the ranked list based on their cosine similarity relative to these dense scores, preserving the ordering of text results while positioning images at comparable relevance levels.

3.7.7 Prompting Strategies for Answer Generation. Answer generation is handled by `VisionGenerator`, an Ollama-backed interface to the `qwen3-vl:8b` vision-language model. When image evidence is present, image files are encoded and passed alongside textual context.

The following prompting strategies are implemented:

- **Standard**: The grounding-focused prompt from System 1, used as the reference configuration.
- **Chain-of-Thought (CoT)**: The model is instructed to perform step-by-step reasoning before producing a final answer, improving performance on multi-step queries.
- **Few-Shot**: A small number of example question–answer pairs are included to demonstrate expected output structure and format consistency.
- **Role**: The model is assigned a domain-specific persona (e.g. financial analyst), guiding tone and reasoning style toward task-relevant outputs.

3.8 System 3 (Agentic mRAG)

3.8.1 Architecture and Pipeline Overview. Architecture and Pipeline Overview. System 3 extends Systems 1 and 2 by introducing agent-based orchestration via LangChain and LangGraph, enabling multiple LLM-backed agents to collaborate through structured decision-making rather than executing a linear pipeline. The system comprises three specialized agents (Query Rewriter, Grader, and Generator), each making autonomous decisions about how to proceed. This design enables controlled experimentation by allowing each agent to reason about its task independently while a graph-based routing mechanism determines the overall control flow. Figure 3 visualizes the agentic architecture.

3.8.2 Framework used. The pipeline is implemented using LangGraph's StateGraph, which models execution as a directed graph with shared state. Each node is a Python function that reads and updates a shared AgenticRAGState object, storing all decisions made, confidence scores, retrieved documents, and reasoning traces throughout the pipeline execution. LangGraph provides built-in state management, typed interfaces (via Pydantic), conditional routing (via router functions that inspect state, and determine next steps), and improved observability for debugging and analysis.

3.8.3 Agent specification & Decision-making. **Query Rewriter Agent**. The Query Rewriter agent determines which of eight query transformation techniques to apply to the user's original question. The agent receives the question and outputs a `QueryRewriterDecision` object with the following fields:

- technique (required): Selected technique from standard, multi_query, rag_fusion, step_back, hyde, query_decomposition, query_rewriting, query_expansion
- Structured explanation of why this technique is appropriate for the question (e.g., "Question requires multiple search perspectives")
- A list of query variants generated by the selected technique

The agent's decision is informed by characteristics of the original question (e.g., complexity, domain specificity). Once the technique is chosen, all rewritten query variants are executed against the retriever

Grader Agent. The Grader agent evaluates whether the retrieved documents are sufficiently relevant to answer the user's question, and determines whether retry should be attempted. The agent inspects the retrieved documents and outputs a `DocumentGrade` object containing:

- relevant (required): Binary decision (yes, no)
- confidence (0.0–1.0): Confidence score in the relevance decision
- num_relevant: Count of documents deemed relevant within the retrieved set
- reasoning: Structured justification for the grade (e.g., "3 of 5 documents directly address the query; threshold met")

The confidence score is central to the retry logic: a high score (e.g., ≥ 0.6) indicates the agent is confident in its assessment, while low confidence triggers potential retry if retry attempts remain.

Generator Agent. The Generator agent selects which prompting strategy to use when generating the final answer from retrieved context. It outputs a `GeneratorDecision` object with:

- strategy (required): Selected strategy from standard, cot, few_shot, role
- role_type (optional): If strategy="role", specifies the expert persona (financial_analyst, researcher, data_analyst, domain_expert, technical_writer)
- reasoning: Justification for strategy choice (e.g., "Query involves multi-step reasoning; CoT strategy recommended")
- confidence (0.0–1.0): Confidence in the answer quality given available context

3.8.4 Conditional Routing and Retry Logic. After grading, a routing function determines whether to retry retrieval or proceed to generation. A retry is triggered if the documents are not relevant or confidence is below a threshold (e.g. 0.6), provided the retry limit (max 1) is not exceeded. On retry, the Query Rewriter is re-invoked with the constraint of avoiding the previously used technique. The loop terminates when sufficient relevance is achieved or retries are exhausted.

3.8.5 State Management & Decision Auditing. All intermediate decisions are stored in the shared AgenticRAGState, including query variants, grading outcomes, selected strategies, and reasoning traces. This enables inspection of the full decision process for debugging and analysis, and serves as input to the evaluation pipeline.

3.9 Metrics

For evaluation, we report metrics for both retrieval quality and generation (answer) quality. Retrieval metrics are computed over top-k ranked results ($k \in \{1, 3, 5\}$), while generation metrics are computed against the ground-truth answer strings.

3.9.1 ***Precision@k (P@k).*** This metric assesses how many of the top-k retrieved documents are actually relevant to the query. A higher precision@k value indicates that the retrieval system is effectively filtering out irrelevant results [17].

3.9.2 ***Recall@k (R@k).*** This metric measures the fraction of all available relevant documents that appear in your top-k retrieved results. It indicates how comprehensively the system captures relevant information within the specified rank cutoff [17].

3.9.3 ***Page Recall (@ k).*** This metric evaluates if any of the top-k retrieved chunks comes from the correct document and overlaps a ground-truth evidence page. It captures whether at least one relevant evidence page is retrieved, reflecting practical retrieval success in long-document QA. This metric helped improved the iterations when developing. *See appendix 10 on notes about this.*

3.9.4 ***Normalized Discounted Cumulative Gain (NDCG@ k).*** This metric evaluates ranking quality by assigning progressively lower importance to results deeper in the ranking, then normalizing against an ideal ranking. It produces a score between 0 and 1 that accounts for both relevance and position, thus making it comparable across different queries [17].

3.9.5 ***Mean Average Precision (MAP).*** This metric calculates the average precision at each position where a relevant document is found, providing an overall ranking quality assessment. It evaluates performance holistically across all queries by rewarding systems that rank relevant documents higher in the result list [17].

3.9.6 ***Mean Reciprocal Rank (MRR).*** This metric quantifies how early the first relevant document appears in the ranked results, with better scores for earlier positions [17].

3.9.7 ***Exact Match (EM).*** This metric measures how often the model's predicted string is identical to the reference string, with higher values indicating that predicted strings exactly match the reference strings more frequently [18].

3.9.8 ***Contains Match.*** Contains Match is a relaxed variant of Exact Match, it evaluates if the ground-truth answer appears as a substring of the predicted answer. It is useful in open-ended

QA where correct answers may appear within longer generated responses.

3.9.9 ***Token F1.*** This metric computes the harmonic mean of precision and recall at the token-level between the generated and reference answers, balancing both metrics to produce a single score that reflects how well tokens are matched between predictions and references.

3.9.10 ***BLEU.*** This metric measures n-gram precision by comparing word sequences between generated and reference text, then applies a brevity penalty to prevent artificially high scores from short outputs. It combines multiple n-gram precision levels into a single score using geometric mean, producing a value between 0 and 1 where higher indicates closer similarity to the reference [19].

3.9.11 ***ROUGE.*** This metric emphasizes recall by measuring how much reference content is captured in the generated text, comparing overlapping word sequences and longest common subsequences. Unlike precision-focused metrics, ROUGE rewards outputs that comprehensively cover reference information, producing a score between 0 and 1 where higher indicates better content coverage [19].

3.9.12 ***Semantic (Textual) Similarity (STS).*** This metric measures how closely two texts align in meaning by representing each text as a vector (embedding) and comparing those vectors using cosine similarity. In this formulation, a score closer to 1 indicates very high semantic similarity, a score around 0 indicates little to no similarity, and higher similarity means the generated text has captured the same underlying meaning as the reference text even if the wording differs [22].

3.10 Ablation Methodology

The ablation study for System 2 is conducted in a sequential and controlled manner to isolate the contribution of each component. Each experiment builds on the best-performing configuration from the previous step.

We begin by comparing text-only and multimodal retrieval, selecting the better-performing approach as the foundation for subsequent experiments. Next, we evaluate chunking strategies, followed by query processing techniques under fixed retrieval and chunking settings. Finally, we assess prompting strategies for answer generation using the selected components.

This stepwise design enables controlled comparison of individual components while progressively constructing the final system. Interactions between components are not exhaustively explored, and alternative combinations may yield different results.

Additionally, an answer-validation mechanism is implemented to post-process and standardize raw LLM outputs (see Section 10).

3.11 Optimization & Time measurements & Reproducibility

Optimization and reproducibility were treated as first-class design goals rather than post-hoc additions. The project uses centralized configuration, explicit cache layers, and staged experiment execution to make ablations tractable under limited compute. This was essential for running many strategy combinations while maintaining fair and repeatable comparisons.

System optimization, runtime measurement, and reproducibility were handled through caching, index reuse, deterministic execution, and configuration-driven workflows. Embeddings and Qdrant indexes are precomputed and reused across runs; at runtime, each pipeline checks whether a populated collection already exists and loads it directly, while indexing is triggered only when needed (force_rebuild option when preprocessing or index settings change). To minimize GPU usage and avoid redundant computation, preprocessed chunk files are persisted, vector indexes are reused unless explicitly rebuilt, and generator calls are cached in SQLite using serialized request payloads and model outputs; generation also uses deterministic decoding settings (low temperature and constrained top-p) to reduce variance and unnecessary reruns. For measurement and reporting, timing is recorded per pipeline phase and exported to CSV, including preprocessing time (PDF extraction and chunking), index build time, and runtime per experiment, with GPU usage reported when applicable. Reproducibility is further supported by deterministic evaluation scripts for retrieval and generation metrics, and documented pipeline steps and run commands in the repository `README` and project scripts.

4 RESULTS

This section reports the quantitative evaluation on the entire test dataset (150 questions). Unless otherwise stated, runs use the same generator family (`qwen3-vl:8b-instruct`) and the same embedding model (`jinaai/jina-clip-v2`). We report both retrieval quality and generation quality, then analyze the ablations that drove the final System 2 configuration.

4.1 System 1 (Baseline)

For System 1, the retrieval performance shows a Precision@1 of 0.5933, indicating that the top-ranked retrieved document is relevant in approximately 59% of the cases. Precision@3 and Precision@5 are 0.2311 and 0.1467 respectively, while Recall@1, Recall@3, and Recall@5 are 0.5933, 0.6933, and 0.7333, showing an increase in coverage as more documents are considered. The overall ranking quality is reflected in an NDCG@5 score of 0.6693, with MAP and MRR both at 0.6478.

For generation performance, the system achieves a Token F1 score of 0.1837, BLEU of 0.1537, ROUGE-1 of 0.2096, ROUGE-2 of 0.0859, and ROUGE-L of 0.2081. The Exact Match score is 0.1333, the Contains Match score is 0.16, and the Semantic Similarity score is 0.7240. These metrics provide an overview of the baseline system's performance across both retrieval and answer generation tasks under the evaluated test conditions.

System	P@1	NDCG@5	Token F1	STS	Exact Match
Baseline	0.5933	0.6693	0.1837	0.7240	0.1333

Table 1: Evaluation results for the baseline system.

4.2 System 2 (Advanced) & Ablations

4.2.1 Text vs. Multimodal Retrieval. We compare a text-only retrieval pipeline with a multimodal retrieval approach incorporating visual signals via cross-modal representations. The results are shown in Table 2.

Multimodal retrieval achieves higher scores across all generative metrics. Token F1 increases from 0.1779 (text-only) to 0.2528, while BLEU improves from 0.1506 to 0.212. ROUGE-1 rises from 0.2043 to 0.2724, and ROUGE-2 from 0.092 to 0.1186. Exact match improves from 0.12 to 0.1867, and semantic similarity from 0.7157 to 0.7263.

Retrieval metrics show smaller differences between the two approaches. Precision@1 increases from 0.86 to 0.8667, while NDCG@5 remains nearly unchanged (0.9040 vs. 0.9043). Similar minor differences are observed for MAP (0.8919 vs. 0.8944) and recall@3 (0.9133 vs. 0.92).

Overall, multimodal retrieval outperforms text-only retrieval on generative metrics, while retrieval performance remains comparable between the two settings.

Modality	P@1	NDCG@5	Token F1	STS	EM
Text-only	0.8600	0.9040	0.1779	0.7157	0.1200
Multi-modal	0.8667	0.9043	0.2528	0.7263	0.1867

Table 2: Modal ablation results in System 2.

4.2.2 Chunking Strategy. Table 3 reports the effect of different chunking strategies on retrieval and generation performance.

For retrieval metrics, fixed-size (2k) and hierarchical chunking achieve the highest scores. Both obtain Precision@1 of 0.8667, while fixed-size (2k) achieves the highest NDCG@5 (0.9043) and MAP (0.8944). Sliding window and semantic chunking show comparable performance, with slightly lower NDCG@5 scores (0.8899 and 0.8861, respectively). Enhanced hierarchical chunking performs worst across retrieval metrics, with Precision@1 dropping to 0.7133 and NDCG@5 to 0.8245.

For generation metrics, fixed-size (2k) performs best across all measures, achieving the highest Token F1 (0.2528), semantic similarity (0.7263), and exact match (0.1867). Other methods show lower and relatively similar scores, with hierarchical (Token F1: 0.2245) and semantic chunking (Token F1: 0.2284) trailing behind. Enhanced hierarchical chunking shows moderate generation performance (Token F1: 0.2300), but remains below fixed-size (2k).

Overall, fixed-size (2k) chunking achieves the strongest performance across both retrieval and generation metrics, while enhanced hierarchical chunking performs worst.

Chunking	P@1	NDCG@5	Token F1	STS	Exact Match
Fixed-size (1k)	0.8533	0.8848	0.2321	0.7154	0.1667
Fixed-size (2k)	0.8667	0.9043	0.2528	0.7263	0.1867
Sliding window	0.8533	0.8899	0.2269	0.7105	0.1600
Semantic	0.8667	0.8861	0.2284	0.7143	0.1600
Hierarchical	0.8667	0.8944	0.2245	0.7149	0.1600
Enhanced hier.	0.7133	0.8245	0.2300	0.7258	0.1733

Table 3: Chunking Strategy ablation results in System 2.

4.2.3 Query Processing. Table 4 compares different query processing techniques.

The standard query achieves the strongest retrieval performance, with the highest Precision@1 (0.8667), NDCG@5 (0.9043), and MAP (0.8944). Rewriting performs similarly, with slightly lower Precision@1 (0.8467) and NDCG@5 (0.8974). Multi-query also shows competitive retrieval performance (P@1: 0.8333, NDCG@5: 0.8745), while expansion and RAG-fusion achieve moderate results. HyDE performs worst across retrieval metrics, with Precision@1 of 0.7000 and NDCG@5 of 0.7403.

For generation metrics, multi-query achieves the highest Token F1 (0.2573), BLEU (0.219), and ROUGE-1 (0.2806). Standard and rewriting follow closely, with Token F1 scores of 0.2528 and 0.2444, respectively. RAG-fusion achieves the highest semantic similarity (0.7283), while exact match is highest for both standard and multi-query (0.1867). HyDE again performs worst across most generation metrics.

Overall, the standard query provides the strongest retrieval performance, while multi-query achieves the highest generation scores. Other query processing techniques show mixed performance across metrics.

Query	P@1	NDCG@5	Token F1	STS	Exact Match
Standard	0.8667	0.9043	0.2528	0.7263	0.1867
Multi-query	0.8333	0.8745	0.2573	0.7270	0.1867
RAG-fusion	0.7667	0.8411	0.2213	0.7283	0.1600
Step-back	0.7533	0.8174	0.2280	0.7142	0.1667
HyDE	0.7000	0.7403	0.1857	0.6992	0.1200
Rewriting	0.8467	0.8974	0.2444	0.7229	0.1800
Decomposition	0.7333	0.8150	0.2265	0.7203	0.1600
Expansion	0.8000	0.8465	0.2383	0.7239	0.1667

Table 4: Query technique ablation in System 2.

4.2.4 Prompting Strategies. Table 5 compares different prompting strategies for answer generation.

CoT achieves the highest Token F1 (0.2658), BLEU (0.2293), ROUGE-1 (0.281), and Exact Match (0.2067). The standard prompt follows, with Token F1 of 0.2528 and Exact Match of 0.1867, and achieves the highest semantic similarity (0.7263).

Few-shot prompting results in lower performance across all metrics (Token F1: 0.1675, Exact Match: 0.1267), while role prompting performs worst overall, with Token F1 of 0.1308 and Exact Match of 0.0867.

Overall, CoT achieves the best performance on most generation metrics, while the standard prompt yields the highest semantic similarity.

Prompting	Token F1	BLEU	STS	Exact Match
Standard	0.2528	0.2120	0.7263	0.1867
CoT	0.2658	0.2293	0.6831	0.2067
Few-shot	0.1675	0.1390	0.5991	0.1267
Role	0.1308	0.1036	0.5698	0.0867

Table 5: Prompt strategy ablation (presentation-aligned results).

4.3 System 3 (Agentic)

Results for the agentic system (System 3) are shown in Table 6. These results indicate that retrieval performance shows a Precision@1 of 0.6066, indicating that the top-ranked retrieved document is relevant in just over 60% of the cases. Precision@3 and Precision@5 are 0.2333 and 0.1426 respectively, while Recall@1, Recall@3, and Recall@5 are 0.6066, 0.7000, and 0.7133. The ranking quality is reflected in an NDCG@5 score of 0.6695, with MAP and MRR both reported as 0.6544.

For generation performance, the system achieves a Token F1 score of 0.1674, a BLEU score of 0.1271, ROUGE-1 of 0.1955, ROUGE-2 of 0.0866, and ROUGE-L of 0.1924. The Exact Match score is 0.1133, the Contains Match score is 0.1133, and the Semantic Similarity score is 0.6562. These values summarize the agentic system's performance across both retrieval and answer generation on the evaluated test set.

System	P@1	NDCG@5	Token F1	STS	Exact Match
Agentic	0.6066	0.6695	0.1674	0.6562	0.1133

Table 6: Evaluation results for the agentic system.

4.4 System 1 vs. System 2 vs. System 3

Table 7 compares the three systems across retrieval and generation metrics.

For retrieval, System 2 achieves the highest performance across all metrics, with Precision@1 of 0.8667, Recall@1 of 0.8667, NDCG@5 of 0.9043, and MAP of 0.8944. System 1 and System 3 obtain lower and similar scores, with Precision@1 of 0.5933 and 0.6067, and Recall@1 of 0.5933 and 0.6066, respectively. Their NDCG@5 scores are also similar (0.6693 and 0.6696).

For generation, System 2 achieves the highest scores on Token F1 (0.2658), BLEU (0.2293), ROUGE-1 (0.281), and Exact Match (0.2067). System 1 shows intermediate performance (Token F1: 0.1837, Exact Match: 0.1333), while System 3 performs lowest across these metrics (Token F1: 0.1675, Exact Match: 0.1133). System 2 also achieves the highest Contains Match score (0.2333). In contrast, System 1 obtains the highest semantic similarity (0.7240), followed by System 2 (0.6831) and System 3 (0.6562).

Overall, System 2 achieves the strongest performance across both retrieval and generation metrics, while System 1 and System 3 show lower and comparable results.

System	P@1	NDCG@5	Token F1	STS	Exact Match
System 1 (Baseline)	0.5933	0.6693	0.1837	0.7240	0.1333
System 2 (Advanced)	0.8667	0.9043	0.2658	0.6831	0.2067
System 3 (Agentic)	0.6067	0.6696	0.1675	0.6562	0.1133

Table 7: End-to-end comparison of baseline, advanced, and agentic systems.

4.5 Efficiency and Runtime

Runtime results reveal important trade-offs. Some advanced settings raise answer quality but substantially increase full evaluation time. Query expansion and RAG-fusion were among the most expensive query techniques, while simpler retrieval paths were significantly faster.

Pipeline	Index Build/Load (s)	Full Evaluation (s)	Total Runtime (s)
System 1 Baseline	45.8012	77.4237	124.2028
System 2 (best retrieval-focused)	44.5680	111.0101	156.2155
System 2 query expansion	50.2205	258.3494	310.1721
System 2 RAG-fusion	50.4469	302.3411	354.6152
System 3	20.7500	1802.5500	1823.3000

Table 8: Representative runtime comparison across configurations.

Overall, the results in Table 8 indicate that the best practical trade-off is achieved by a non-agentic, carefully tuned System 2 configuration, which balances retrieval effectiveness, answer quality, and runtime cost more effectively than both the baseline and the agentic System 3.

5 DISCUSSION

5.1 When Multimodal Retrieval Helped and Failed

Multimodal retrieval improves answer quality compared to text-only retrieval, particularly for queries that require information beyond plain text. As shown in Table **??**, the largest gains are observed in generation metrics such as Token F1 and Exact Match, while retrieval metrics remain largely unchanged.

A key factor behind these improvements is the nature of the queries. Many questions require information that is not fully captured in textual form, such as figures, tables, or diagrams. For example, the question "In the pipeline diagram of the RAR model, which type of organism is used as the input case?" relies on information presented in a diagram rather than in the accompanying text. In such cases, text-only retrieval cannot provide sufficient context, whereas multimodal retrieval enables access to the required visual information.

In contrast, for queries where all relevant information is explicitly stated in the text, multimodal retrieval provides limited benefit. This explains why retrieval metrics remain similar across both approaches, as both methods often retrieve the same documents but differ in the completeness of the retrieved context.

Overall, multimodal retrieval is most beneficial for visually grounded queries, while offering limited gains for purely text-based questions.

5.2 Impact of Preprocessing and Chunking

Preprocessing and chunking jointly determine how effectively contextual information is preserved before embedding and retrieval. Removing noisy elements (e.g., headers and footers) improves embedding quality and enables more coherent chunk boundaries. This motivated the use of structured extraction via `docling`, highlighting the strong dependency between preprocessing quality and chunking performance.

These results also highlight that chunking performance is closely tied to preprocessing quality, as cleaner and more structured input enables more coherent chunk boundaries.

A key observation from the chunking experiments (Table 3) is that Fixed-size (2k) consistently achieves the strongest overall performance, reaching the highest NDCG@5 (0.9043) and MAP

(0.8944), while also obtaining the best generation performance in terms of Token F1 (0.2528) and semantic similarity (0.7263). This suggests that increasing chunk size improves retrieval effectiveness by reducing the likelihood of splitting semantically coherent passages across multiple chunks.

This effect is further supported by the chunk length distribution analysis (see Figure 4 in the Appendix). As shown in the distribution plot, Fixed-size (2k) produces substantially larger and more consistent chunks compared to smaller or structurally driven strategies. In contrast, Fixed-size (1k) and sliding-window approaches generate a higher number of boundary cuts around dense textual regions, increasing the probability that relevant context is partially split between chunks.

This fragmentation degrades retrieval quality by distributing relevant information across multiple embeddings instead of a single chunk. Hierarchical and semantic strategies partially mitigate this by respecting document structure, but their variable chunk sizes introduce inconsistency in context granularity, which is reflected in their slightly lower retrieval scores.

These improvements in context preservation also translate into stronger downstream performance. For example, for the query "How many different Chinese characters are shown in the slide?", the system correctly retrieves the relevant visual context and produces the exact answer (3). This indicates that well-structured chunks preserve both textual and visual references required for multimodal reasoning. Similarly, for visually grounded queries such as identifying objects in images (e.g., recognizing the Eiffel Tower in a demonstration screenshot), the multimodal pipeline enables accurate answer generation.

Overall, the results indicate that chunking quality is not only determined by semantic structure, but also by the preservation of complete contextual units and the quality of preprocessing. In this setup, larger fixed-size chunks combined with structured preprocessing provide a better balance between noise and completeness, leading to improved retrieval stability and downstream generation performance.

5.3 Impact of Query Processing Strategies

Advanced query processing techniques do not consistently outperform the standard query baseline, with no method providing a clear overall improvement.

A key observation is the trade-off between retrieval and generation performance. The standard query achieves the highest retrieval scores (P@1: 0.8667, NDCG@5: 0.9043, MAP: 0.8944), indicating that the original query formulation is already well aligned with the underlying document representations. In contrast, multi-query achieves the highest generation scores (Token F1: 0.2573), but with slightly reduced retrieval performance. This suggests that generating multiple query variants can improve coverage of relevant information, but may also introduce less precise or partially redundant queries.

Several query transformation techniques, including rewriting, decomposition, and expansion, show performance close to the baseline but do not provide consistent improvements. This indicates that modifying the original query can introduce small variations without reliably improving retrieval quality. In some cases, these transformations may alter or omit important contextual relationships present in the original query, leading to reduced retrieval effectiveness.

HyDE performs worst across both retrieval and generation metrics. This may be due to the increased reliance on the language model to generate a high-quality hypothetical document. When the generated representation is inaccurate or only partially aligned with the true information need, retrieval quality degrades significantly. Compared to other methods, HyDE introduces a larger transformation step, making it more sensitive to model limitations.

The performance of these techniques is also influenced by the capacity of the underlying language model. The use of a relatively small model (`qwen3-vl:8b-instruct`) may limit the quality of generated query transformations, particularly for more complex methods such as HyDE and decomposition. Larger models may produce more accurate and context-preserving reformulations, potentially improving the effectiveness of these approaches.

Finally, the computational cost of advanced query processing must be considered. Methods such as multi-query require additional language model calls per query, increasing latency and resource usage. Given that the observed improvements in generation metrics are relatively small, these costs may outweigh the benefits in practical settings.

Overall, the results indicate that while advanced query processing can introduce variation in performance, the standard query provides the most reliable balance between retrieval quality, generation performance, and computational efficiency in this setup.

5.4 Impact of Prompting Strategies

The results show that prompting strategies have a substantial impact on generation performance, with clear differences across approaches.

CoT prompting achieves the strongest overall performance, with the highest Token F1 (0.2658), BLEU (0.2293), ROUGE-1 (0.281), and Exact Match (0.2067). This improvement is consistent with the nature of the dataset, which includes many questions requiring multi-step reasoning rather than direct extraction. For example, questions that involve identifying entities across multiple sections and computing differences benefit from explicit intermediate reasoning steps. CoT enables the model to structure this reasoning process, leading to more accurate answers.

In contrast, the standard prompt performs slightly worse on most generation metrics but achieves the highest semantic similarity (0.7263). This suggests that while answers are often semantically close to the reference, they may lack the structured reasoning needed for exact correctness in more complex queries.

Few-shot prompting performs significantly worse across all metrics. One likely reason is the diversity of the dataset, which spans multiple domains and question types. A small set of static examples may not generalize well across such varied queries, limiting their effectiveness. Additionally, few-shot prompting primarily influences answer formatting and style rather than improving the model's reasoning capabilities, which may explain the limited gains.

Role prompting performs worst overall. This can be attributed to the mismatch between a fixed role (e.g., financial analyst) and the heterogeneous nature of the dataset. While a specialized role may

improve performance on a narrow subset of domain-specific questions, it introduces bias that degrades performance on unrelated queries. Furthermore, role-based prompts tend to produce more verbose or stylistically constrained outputs, which can negatively impact metrics such as Exact Match and Token F1, which favor concise and precise answers.

Another contributing factor across prompting strategies is output control. Both role and few-shot prompting introduce additional instructions that can conflict with strict output constraints, increasing the likelihood of deviations in formatting or verbosity. In contrast, the standard and CoT prompts maintain stronger alignment with the required output format.

Overall, CoT provides the most effective balance between reasoning capability and answer accuracy, particularly for multi-step questions. Simpler prompting strategies remain competitive for direct retrieval-style queries, while more complex or stylistically constrained prompts may introduce overhead without improving performance.

5.5 Agentic vs. Non-Agentic Trade-off

The comparison between non-agentic systems (System 1 and System 2) and the agentic system (System 3) highlights a clear trade-off between complexity and performance. System 2 achieves the best results across both retrieval and generation metrics, while System 3 performs similarly to the baseline (System 1) despite significantly higher computational cost. In particular, System 3 requires approximately 12.16 seconds per query, and between 4 (best case) and 8 (worst case) LLM calls, compared to 1–2 calls for the non-agentic systems. This results in a clear efficiency–effectiveness trade-off, where increased computational cost (in terms of latency and number of LLM calls) does not translate into improved retrieval or generation performance.

This behavior can be explained by the structure of the agentic pipeline. System 3 relies on a sequence of dependent decisions, where each agent builds on the output of the previous step. As a result, an early suboptimal decision (e.g., an ineffective query rewriting strategy) propagates through the pipeline and degrades overall performance. While a retry mechanism is present, recovery is limited and does not fully mitigate these cascading errors. For example, for the query "How many times does 'Barclays' appear on page 4?", the retriever fails to retrieve any relevant documents in both attempts. The grader correctly identifies the lack of relevant evidence, yet the system still proceeds to answer generation, producing the final answer "0". Such cases illustrate how failures in retrieval propagate through the pipeline and lead to incorrect or incomplete final outputs, despite the presence of intermediate validation steps.

In addition, the agents operate without access to historical performance feedback (apart from notifying the Query Rewriter which technique *not* to use during a retry), relying solely on per-query reasoning. This limits their ability to consistently select effective strategies. Furthermore, the use of a relatively small model (`qwen3-vl:8b-instruct`) for all agent roles may further constrain decision quality, particularly for tasks requiring meta-reasoning about retrieval and prompting strategies.

In contrast, the non-agentic systems benefit from simplicity and stability. System 2, in particular, uses a fixed and carefully tuned configuration, allowing its components to be jointly optimized. This avoids the variability introduced by dynamic decision-making and leads to more reliable performance across queries.

Overall, the results suggest that agentic systems are not inherently superior, and their effectiveness depends on the task. On MMDocIR, where queries are relatively structured and homogeneous, a well-tuned fixed pipeline is sufficient and more efficient. In more diverse or open-domain settings, however, the adaptive capabilities of agentic systems may provide greater benefits, potentially justifying their additional complexity and cost.

5.6 Baseline vs. Advanced vs. Agentic

The comparison between System 1, System 2, and System 3 highlights the relative importance of retrieval quality, system design complexity, and component optimization. As shown in Table 7, System 2 consistently outperforms both the baseline and the agentic system across most metrics.

The transition from System 1 to System 2 demonstrates the impact of systematic improvements in retrieval and generation components. Precision@1 increases from 0.5933 to 0.8667, while Token F1 improves from 0.1837 to 0.2658. These gains reflect the cumulative effect of optimized chunking, query processing, multimodal retrieval, and prompting strategies.

In contrast, System 3 does not achieve similar improvements despite its more complex architecture. Its retrieval performance (P@1: 0.6067, NDCG@5: 0.6696) remains close to the baseline, and its generation performance (Token F1: 0.1675) is lower than both System 1 and System 2. This indicates that increased architectural flexibility does not automatically translate into better performance.

Across the three systems, a clear pattern emerges: improvements in retrieval quality directly translate into better downstream generation. System 2 benefits from strong and stable retrieval, while System 3 introduces variability that reduces overall effectiveness. These results reinforce the importance of retrieval robustness as the primary driver of end-to-end performance in this task.

5.7 Efficiency vs. Effectiveness

The results reveal a clear trade-off between performance gains and computational cost. While System 2 achieves the best overall effectiveness, it also introduces moderate increases in runtime compared to the baseline. More advanced configurations within System 2, such as query expansion and RAG-fusion, further increase computational cost without providing consistent improvements in performance.

As shown in Table 8, simple configurations offer the best balance between efficiency and effectiveness. The baseline system completes evaluation in approximately 124 seconds, while the best-performing System 2 configuration requires around 156 seconds. In contrast, more complex query strategies increase total runtime substantially, exceeding 300 seconds in some cases.

The agentic system represents the extreme end of this trade-off. With a total runtime of over 1800 seconds, it is an order of magnitude slower than the other systems. This is primarily due to multiple LLM calls per query and sequential decision steps. Despite

this cost, System 3 does not achieve better performance, making it inefficient in this setting.

Overall, the results suggest that moderate increases in complexity can improve performance when carefully controlled, but excessive complexity leads to diminishing returns. Efficient system design requires balancing retrieval quality, generation performance, and computational cost, rather than optimizing any single dimension in isolation.

5.8 Challenges, Limitations & Future Work

This study has several limitations related to retrieval design, model capacity, and system architecture.

First, retrieval quality remains the primary bottleneck. Although improvements in chunking and multimodal retrieval significantly increased performance, errors in retrieval propagate directly to generation. The sequential ablation design isolates component effects, but also highlights that downstream improvements cannot compensate for missing or incomplete context.

Second, the effectiveness of several advanced techniques is constrained by the underlying model capacity. The system relies on `qwen3-vl:8b-instruct` for both generation and agent decisions, which may limit the quality of query transformations, multimodal reasoning, and agent-level meta-decisions. More complex techniques such as HyDE or agentic strategy selection are particularly sensitive to this limitation.

Third, the multimodal retrieval pipeline depends on a binary visual routing step and a fixed retrieval budget split. While this design simplifies experimentation, it may not optimally allocate retrieval capacity across different query types. Queries requiring both textual and visual evidence may benefit from more adaptive or learned fusion strategies.

Fourth, the agentic system highlights challenges in multi-step decision pipelines. Agent decisions are made without access to historical performance or learning signals, relying solely on per-query reasoning. This leads to error propagation when early decisions are suboptimal, with limited recovery despite the retry mechanism.

Finally, while the experimental setup ensures reproducibility through deterministic decoding, caching, and index reuse, it also constrains exploration. Hyperparameters such as chunk size, prompt design, and query strategies are evaluated in isolation rather than jointly optimized, which may limit overall system performance.

Future work can address these limitations in several ways. First, adaptive multimodal routing and learned fusion mechanisms could improve alignment between query type and retrieval strategy. Second, larger or fine-tuned models could improve both reasoning and retrieval alignment, particularly for complex query transformations and agent decisions. Third, agent policies could be enhanced through feedback-driven or reinforcement-based learning to improve decision consistency. Finally, more detailed error analysis—especially for visually grounded and multi-hop queries—could guide targeted improvements in both retrieval and generation.

6 CONCLUSION

This project investigated multimodal retrieval-augmented generation as a sequence of design choices evaluated through controlled ablation. Three systems were implemented and compared: a baseline retrieval pipeline (System 1), an advanced multimodal pipeline with systematic component optimization (System 2), and an agentic pipeline with dynamic decision-making (System 3).

The main finding is that a carefully tuned non-agentic system provides the strongest overall performance. System 2 achieves substantial improvements over the baseline across both retrieval and generation metrics, demonstrating the effectiveness of sequential optimization of chunking, query processing, multimodal retrieval, and prompting strategies. In contrast, the agentic system introduces additional complexity without improving performance, highlighting the challenges of multi-step decision pipelines under limited model capacity.

The ablation results show that not all advanced techniques yield consistent gains. Retrieval-focused improvements, particularly chunking strategy and multimodal integration, have the largest impact on end-to-end performance. Query transformation techniques produce mixed results, while prompting strategies such as Chain-of-Thought improve generation quality for more complex queries.

Overall, the results emphasize that retrieval quality is the dominant factor in multimodal RAG systems. Effective system design depends on balancing retrieval robustness, generation quality, and computational efficiency, rather than increasing architectural complexity alone.

7 PROJECT REFLECTIONS

A key takeaway from this project is that retrieval quality dominates end-to-end system performance. Improvements to generation alone had limited impact when retrieval was weak, while even relatively simple retrieval enhancements consistently improved answer quality. This reinforces the importance of focusing on retrieval as the primary optimization target in RAG systems.

A second important lesson was the value of strong engineering practices. The use of centralized configuration, caching of embeddings and generation outputs, and reuse of vector indexes made it possible to conduct large-scale ablation studies under limited computational resources. Combined with deterministic evaluation and controlled experimental design, this enabled fair and reproducible comparisons across multiple system configurations.

The project also highlighted that increased architectural complexity does not necessarily lead to better performance. While the agentic system provided flexibility, interpretability, and structured decision-making, it introduced additional failure points and higher computational cost. In this setup, a simpler, well-optimized pipeline proved more effective and reliable.

Finally, the sequential ablation approach proved valuable for isolating the contribution of individual components. However, it also revealed that interactions between components can be non-trivial, suggesting that future work should explore joint optimization strategies rather than strictly stage-wise tuning.

If extended further, the project would benefit from improved agent decision policies, adaptive multimodal routing, and deeper error analysis across different query types. These directions would help address current limitations and further improve system robustness and generalization.

REFERENCES

[1] Patrick Lewis, Ethan Perez, Aleksandara Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.

[2] Unstructured. *Unstructured Documentation*. https://docs.unstructured.io/. Accessed April 2026.

[3] Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vaghefi, Peter Sheridan, Ingmar Kramer, and Peter W. J. Staar. *Docling Technical Report*. arXiv:2408.09869, 2024.

[4] Qdrant. *Qdrant Documentation*. https://qdrant.tech/documentation/. Accessed April 2026.

[5] Jina AI. *Jina CLIP v2 Model Documentation / Model Card*. https://huggingface.co/jinaai/jina-clip-v2. Accessed April 2026.

[6] BAAI. *bge-large-en-v1.5 Model Card*. https://huggingface.co/BAAI/bge-large-en-v1.5. Accessed April 2026.

[7] Ollama. *Ollama Documentation*. https://github.com/ollama/ollama-python. Accessed April 2026.

[8] Stephen Robertson and Hugo Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Now Publishers Inc, volume 4, 2009.

[9] Gordon V. Cormack, Charles L.A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[10] Kuicai Dong, Yujing Chang, Xin Deik Goh, Dexun Li, Ruiming Tang, and Yong Liu. *MMDocIR: Benchmarking Multi-Modal Retrieval for Long Documents*. arXiv preprint arXiv:2501.08828, 2025. https://arxiv.org/abs/2501.08828. Accessed April 2026.

[11] Tejpal Abhyuday. *Mastering RAG: From Fundamentals to Advanced Query Transformation Techniques (Part 1)*. Medium, 2024. https://medium.com/@tejpal.abhyuday/mastering-rag-from-fundamentals-to-advanced-query-transformation-techniques-part-1-a1fee8823806. Accessed April 2026.

[12] Adrian H. Raudaschl. *Forget RAG, the Future is RAG-Fusion: The Next Frontier of Search*. Towards Data Science (Medium), 2023. https://medium.com/data-science/forget-rag-the-future-is-rag-fusion-1147298d8ad1. Accessed April 2026.

[13] Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V. Le, and Denny Zhou. *Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models*. arXiv preprint arXiv:2310.06117, 2024. https://arxiv.org/abs/2310.06117. Accessed April 2026.

[14] Paul J. L. Ammann, Jonas Golde, and Alan Akbik. *Question Decomposition for Retrieval-Augmented Generation*. arXiv preprint arXiv:2507.00355, 2025. https://arxiv.org/abs/2507.00355. Accessed April 2026.

[15] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. *Query Rewriting in Retrieval-Augmented Large Language Models*. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5303–5315, Association for Computational Linguistics, 2023. https://aclanthology.org/2023.emnlp-main.322/. Accessed April 2026.

[16] Sahin Ahmed. *Query Expansion in Enhancing Retrieval-Augmented Generation (RAG)*. Medium, Nov 2024. https://medium.com/@sahin.samia/query-expansion-in-enhancing-retrieval-augmented-generation-rag-d41153317383. Accessed April 2026.

[17] GeeksforGeeks. *Evaluation Metrics for Retrieval-Augmented Generation (RAG) Systems*. https://www.geeksforgeeks.org/nlp/evaluation-metrics-for-retrieval-augmented-generation-rag-systems/. Accessed April 2026.

[18] IBM. *Exact Match Metric*. https://www.ibm.com/docs/en/watsonx/w-and-w/2.1.0?topic=metrics-exact-match. Accessed April 2026.

[19] GeeksforGeeks. *Understanding BLEU and ROUGE Score for NLP Evaluation*. https://www.geeksforgeeks.org/nlp/understanding-bleu-and-rouge-score-for-nlp-evaluation/. Accessed April 2026.

[20] Spencer Porter. *Understanding Cosine Similarity and Word Embeddings*. Medium, 2023. https://spencerporter2.medium.com/understanding-cosine-similarity-and-word-embeddings-dbf19362a3c. Accessed April 2026.

[21] IBM. *What Is LangChain?*. https://www.ibm.com/think/topics/langchain. Accessed April 2026.

[22] Conor Bronsdon. *Semantic Textual Similarity Metric Guide for AI Applications*. Galileo AI Blog, Nov 2025. https://galileo.ai/blog/semantic-textual-similarity-metric. Accessed April 2026.

8 APPENDIX: ADDITIONAL FIGURES

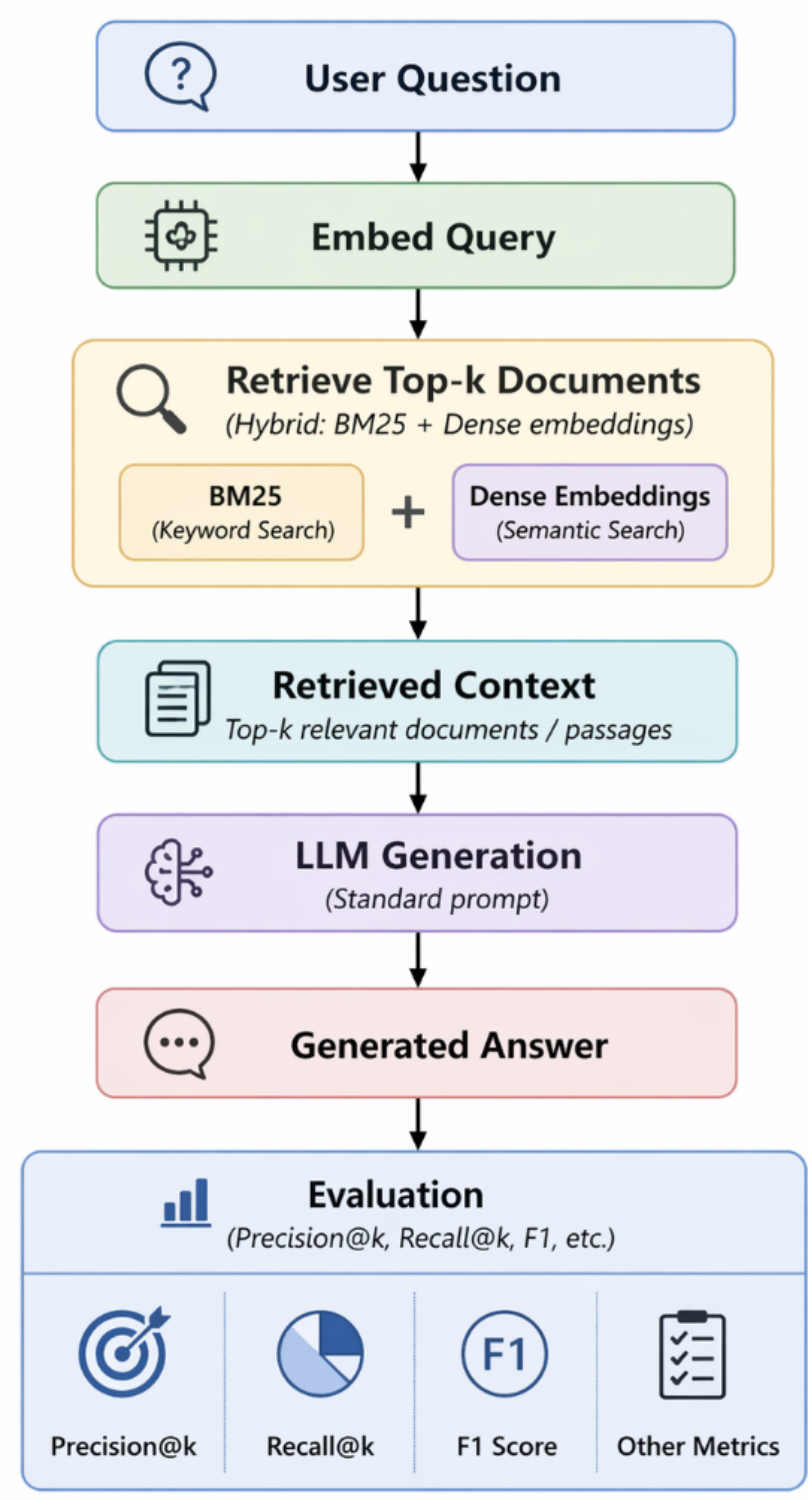

Figure 1: System 1 baseline architecture.

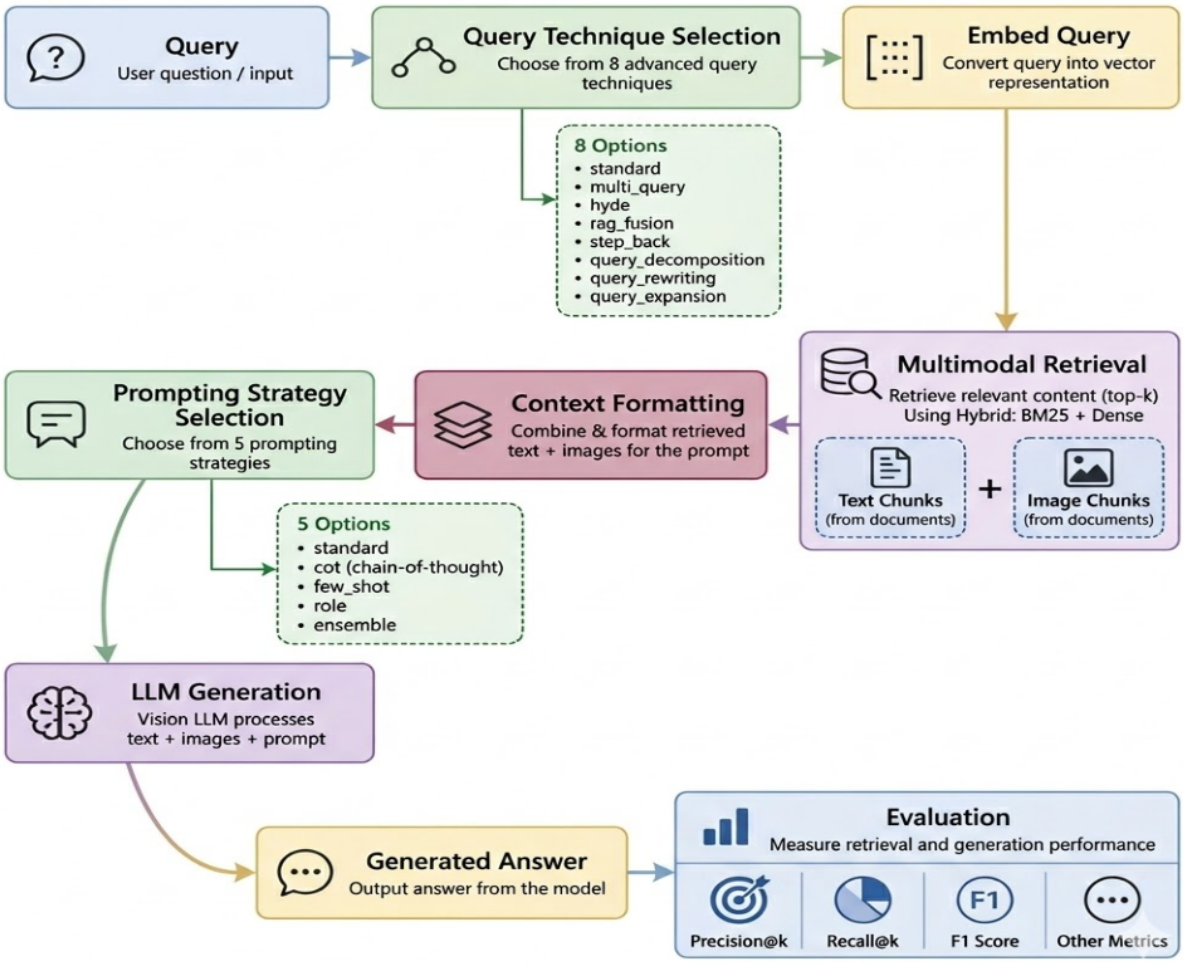

Figure 2: System 2 advanced mRAG architecture.

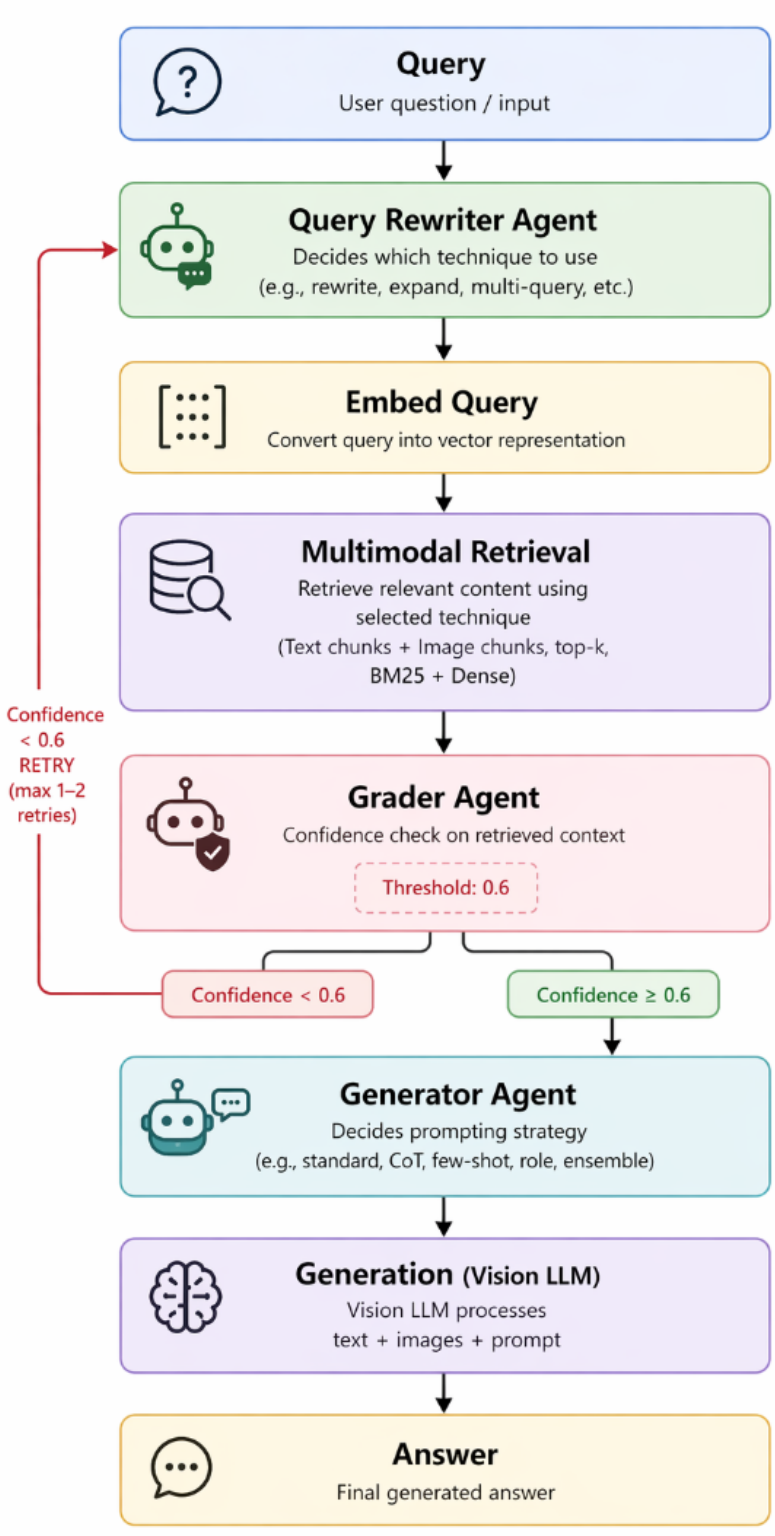

Figure 3: System 3 agentic mRAG architecture.

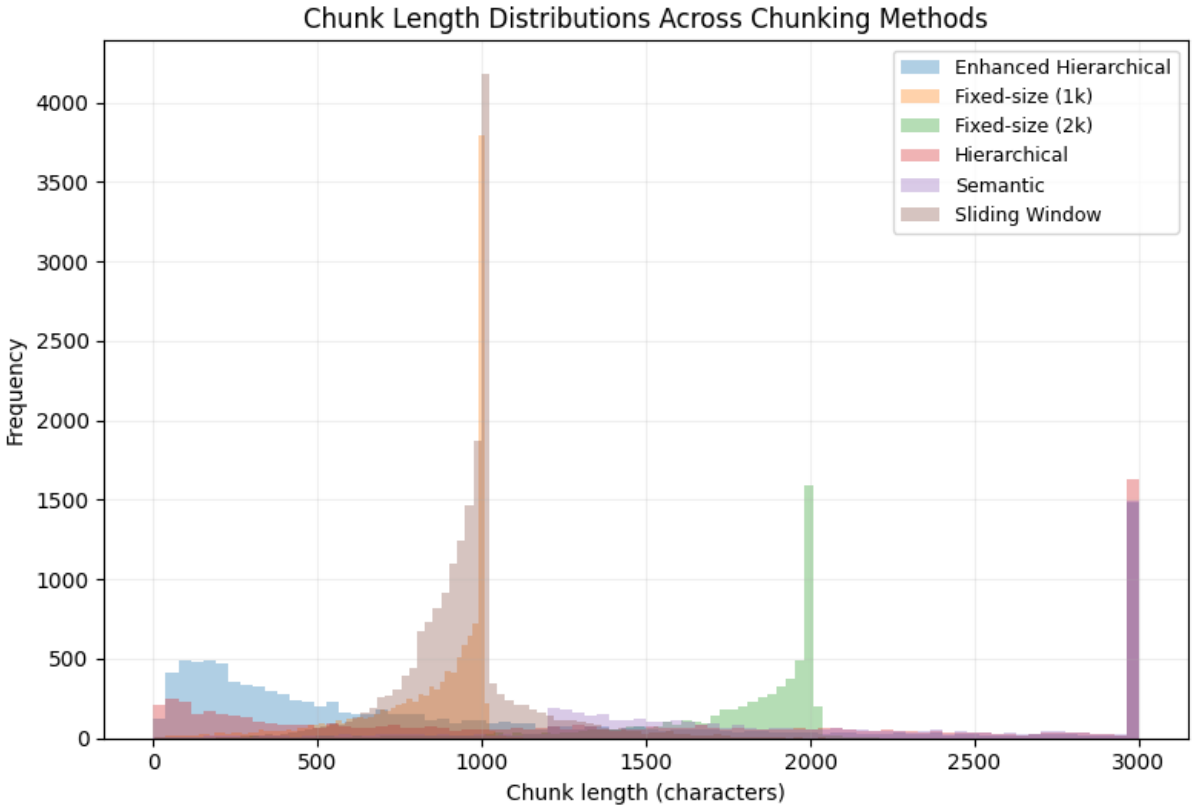

Figure 4: Chunk size distribution across chunking strategies.

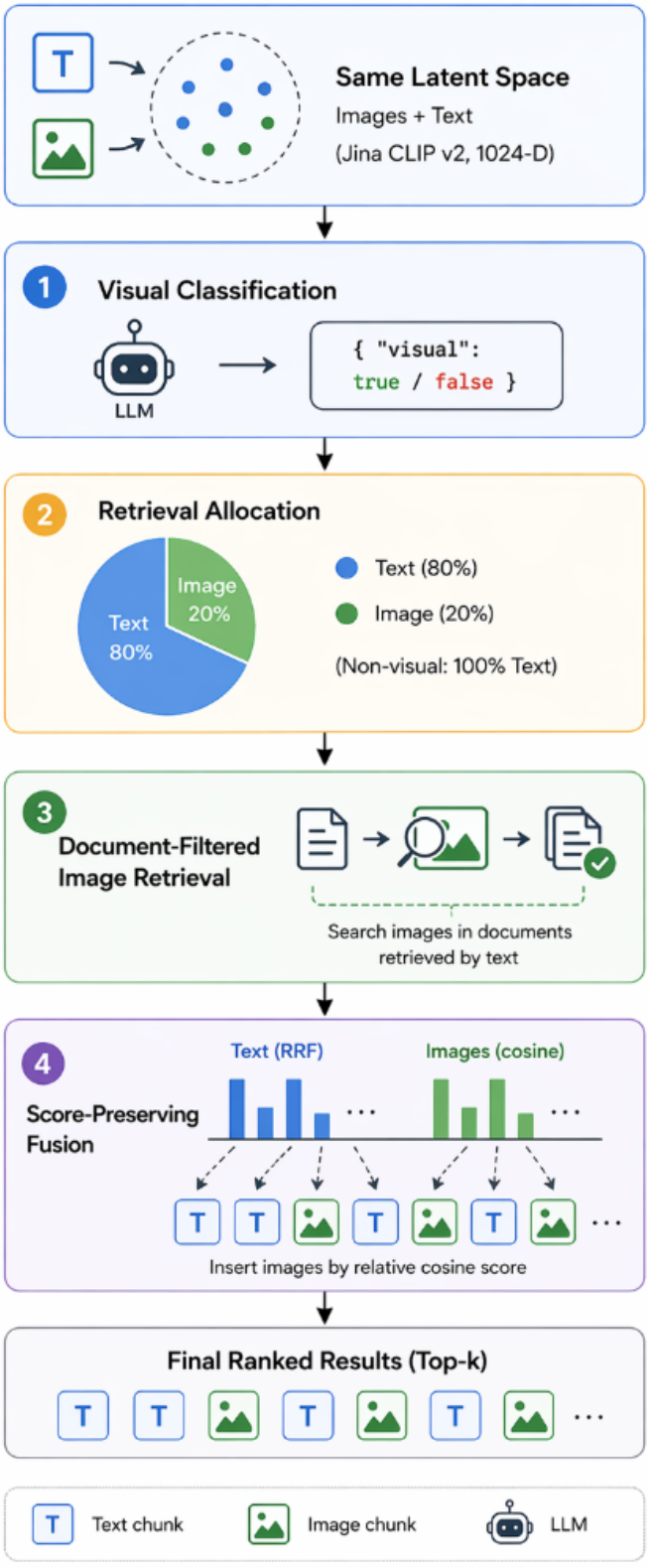

Figure 5: Multimodal retrieval system view used in System 2 experiments.

9 APPENDIX: ADDITIONAL TABLES

The provided tables below are split up across various chunking strategies, query techniques and prompting strategies. Tables for performance of the various systems are also shown. The tables are are segmented by Retrieval metrics and Generation metrics.

9.1 Tables for Modality.

Modality	Token F1	BLEU	ROUGE-1	ROUGE-2	ROUGE-L	Exact Match	Contains Match	Semantic Similarity
Text-only	0.1779	0.1506	0.2043	0.092	0.2081	0.1200	0.1400	0.7157
Multi-modal	0.2528	0.2120	0.2724	0.1186	0.2721	0.1867	0.2200	0.7263

Modality	P@1	P@3	P@5	R@1	R@3	R@5	NDCG@1	NDCG@3	NDCG@5	MAP	MRR
Text-only	0.8600	0.3044	0.188	0.86	0.9133	0.9400	0.8600	0.8928	0.9040	0.8919	0.8919
Multi-modal	0.8667	0.3067	0.1867	0.8667	0.9200	0.9333	0.8667	0.8986	0.9043	0.8944	0.8944

9.2 Tables for Chunking Strategies.

Chunking	Token F1	BLEU	ROUGE-1	ROUGE-2	ROUGE-L	Exact Match	Contains Match	Semantic Similarity
Fixed (1k)	0.2321	0.193	0.2468	0.1069	0.2459	0.1667	0.2067	0.7154
Fixed (2k)	0.2528	0.212	0.2724	0.1186	0.2721	0.1867	0.22	0.7263
Sliding Window	0.2269	0.1862	0.2525	0.115	0.2493	0.16	0.2267	0.7105
Semantic	0.2284	0.1891	0.2478	0.1187	0.2446	0.16	0.2067	0.7143
Hierarchical	0.2245	0.1896	0.2504	0.1067	0.251	0.16	0.2067	0.7149
Enhanced Hier.	0.23	0.2033	0.2554	0.1311	0.2569	0.1733	0.1933	0.7258

Chunking	P@1	P@3	P@5	R@1	R@3	R@5	NDCG@1	NDCG@3	NDCG@5	MAP	MRR
Fixed (1k)	0.8533	0.3	0.1813	0.8533	0.9	0.9067	0.8533	0.8819	0.8848	0.8772	0.8772
Fixed (2k)	0.8667	0.3067	0.1867	0.8667	0.92	0.9333	0.8667	0.8986	0.9043	0.8944	0.8944
Sliding Window	0.8533	0.3022	0.1827	0.8533	0.9067	0.9133	0.8533	0.887	0.8899	0.8817	0.8817
Semantic	0.8667	0.2978	0.18	0.8667	0.8933	0.9	0.8667	0.8835	0.8861	0.8813	0.8813
Hierarchical	0.8667	0.3044	0.1827	0.8667	0.9133	0.9133	0.8667	0.8944	0.8944	0.8878	0.8878
Enhanced Hier.	0.7133	0.2933	0.18	0.7133	0.88	0.9	0.7133	0.8159	0.8245	0.7983	0.7983

9.3 Tables for Query Techniques.

Query	Token F1	BLEU	ROUGE-1	ROUGE-2	ROUGE-L	Exact Match	Contains Match	Semantic Similarity
Standard	0.2528	0.212	0.2724	0.1186	0.2721	0.1867	0.22	0.7263
HyDe	0.1857	0.1487	0.2189	0.0963	0.2156	0.12	0.1533	0.6992
Multi-query	0.2573	0.219	0.2806	0.1283	0.2795	0.1867	0.2267	0.727
step-back	0.228	0.1914	0.2465	0.1152	0.245	0.1667	0.2067	0.7142
Rag-fusion	0.2213	0.1894	0.2523	0.1269	0.2508	0.16	0.2	0.7283
Rewriting	0.2444	0.2069	0.2787	0.1354	0.2782	0.18	0.22	0.7229
Decomposition	0.2265	0.1865	0.2495	0.1176	0.2485	0.16	0.1933	0.7203
Expansion	0.2383	0.199	0.2574	0.1197	0.2545	0.1667	0.2133	0.7239

Query	P@1	P@3	P@5	R@1	R@3	R@5	NDCG@1	NDCG@3	NDCG@5	MAP	MRR
Standard	0.8667	0.3067	0.1867	0.8667	0.9200	0.9333	0.8667	0.8986	0.9043	0.8944	0.8944
HyDe	0.7000	0.2556	0.1533	0.7000	0.7667	0.7667	0.7000	0.7403	0.7403	0.7311	0.7311
Multi-query	0.8333	0.3000	0.1813	0.8333	0.9000	0.9067	0.8333	0.8719	0.8745	0.8636	0.8636
Step-back	0.7533	0.2867	0.1733	0.7533	0.8600	0.8667	0.7533	0.8145	0.8174	0.8006	0.8006
Rag-fusion	0.7667	0.2956	0.1800	0.7667	0.8867	0.9000	0.7667	0.8354	0.8411	0.8211	0.8211
Rewriting	0.8467	0.3089	0.1867	0.8467	0.9267	0.9333	0.8467	0.8945	0.8974	0.8850	0.8850
Decomposition	0.7333	0.2844	0.1760	0.7333	0.8533	0.8800	0.7333	0.8038	0.8150	0.7930	0.7930
Expansion	0.8000	0.2911	0.1760	0.8000	0.8733	0.8800	0.8000	0.8436	0.8465	0.835	0.8350

9.4 Tables for Prompt Strategies.

Prompt	Token F1	BLEU	ROUGE-1	ROUGE-2	ROUGE-L	Exact Match	Contains Match	Semantic Similarity
Standard	0.2528	0.212	0.2724	0.1186	0.2721	0.1867	0.22	0.7263
CoT	0.2658	0.2293	0.281	0.1074	0.2806	0.2067	0.2333	0.6831
Role	0.1308	0.1036	0.1491	0.0773	0.1481	0.0867	0.1667	0.5698
Few-shot	0.1675	0.139	0.185	0.0842	0.1862	0.1267	0.1467	0.5991

9.5 Tables for System Comparison

System	Token F1	BLEU	ROUGE-1	ROUGE-2	ROUGE-L	Exact Match	Contains Match	Semantic Similarity
System 1 (Baseline)	0.1837	0.1537	0.2096	0.0859	0.2081	0.1333	0.16	0.724
System 2 (Advanced)	0.2658	0.2293	0.281	0.1074	0.2806	0.2067	0.2333	0.6831
System 3 (Agentic)	0.1674	0.1271	0.1955	0.0866	0.1924	0.1133	0.1133	0.6562

System	P@1	P@3	P@5	R@1	R@3	R@5	NDCG@1	NDCG@3	NDCG@5	MAP	MRR
System 1 (Baseline)	0.5933	0.2311	0.1467	0.5933	0.6933	0.7333	0.5933	0.6521	0.6693	0.6478	0.6478
System 2 (Advanced)	0.8667	0.3067	0.1867	0.8667	0.92	0.9333	0.8667	0.8986	0.9043	0.8944	0.8944
System 3 (Agentic)	0.6066	0.2333	0.1426	0.6066	0.7	0.7133	0.6066	0.6638	0.6695	0.6544	0.6544

10 APPENDIX: ADDITIONAL NOTES & OBSERVATIONS

10.1 About Page Recall (@ k) metric

We have implemented the Page Recall (@ k) metric in our code implementation, however we did not include this metric in any of the resulting tables presented here. This decision is because of the fact that the computed Page Recall (@ k) values for all evaluations done across all systems resulted in 0.0. This can be explained by the test dataset on which we evaluated on does not have page number metadata associated with it (unlike the train dataset which does have page metadata). We therefore don't find it necessary to include the results for this metric in the main result tables, but is nonetheless worth noting.

10.2 About `answer_validator.py` logic

Notice that there is a file `answer_validator.py` in the `/generation` directory of the project codebase, containing logic for validating and reformatting raw answers acquired from the LLM, attempting to make sure that the generated answers match expected output format. Primarily we consider:

- Count questions: ensure output is an integer
- Proportion (percentage) questions: ensure output is percentage with units
- List questions: ensure output is properly formatted list
- Text questions: basic non-empty validation

The motivation behind the following validation logic is that during development, we observed that the LLM sometimes outputs answers in unexpected formats (e.g., "2 students" instead of "2", or "27 percent" instead of "27%"). This validation layer is designed to catch common formatting issues and correct them when possible, while also providing feedback on any validation failures.

From a metrics evaluation standpoint, particularly for generation metrics like Exact Match or F1, having a consistent answer format is crucial. For example, if the ground truth answer is "27%" but the model outputs "27 percent", a strict Exact Match would fail even though the answer is semantically correct. By validating and correcting the format to a standard representation (e.g., always using "27%"), we can ensure that our evaluation metrics more accurately reflect the model's performance rather than being skewed by formatting inconsistencies.

In other words, doing this validation and correction step allows us to better assess the true quality of the generated answers, rather than penalizing the model for minor formatting variations that do not affect the overall correctness of the answer.

When questioned whether this is considered fair, it's important to note that the goal of this validation layer is not to give the model an unfair advantage, but rather to ensure that we are evaluating the model's understanding and reasoning capabilities rather than its ability to guess the exact formatting of the answer.

That is why in the code, we separate "raw" LLM outputs and "validated" outputs for metrics calculations. Specifically, for ROUGE, BLEU and Token F1 we do *not* apply the "validation" logic on these metrics, while the other generation metrics (exact_match, contains_match, semantic_similarity) does use these "validated" answers.

11 APPENDIX: NOTES ON REPRODUCIBILITY

11.1 Environment details

All experiments were conducted on a shared university GPU server accessed remotely via API. The Ollama inference server is hosted at `https://ollama.ux.uis.no` and was accessed over HTTPS using a bearer token loaded from a `.env` file. The vector database (Qdrant) runs in-memory via `qdrant-client`'s embedded in-process mode (`VECTOR_DB_MODE = "local"`), with storage persisted to disk at `src/local_qdrant/`. No external database process or Docker container is required; Qdrant initialises automatically when the pipeline starts.

Python 3.10+ was used throughout. All dependencies are pinned in `requirements.txt`. The version-sensitive packages are listed in Table 9.

Table 9: Key Python dependency versions.

Package	Version
`transformers`	4.51.3
`FlagEmbedding`	1.2.11
`sentence-transformers`	latest at install time
`open_clip_torch`	latest at install time
`qdrant-client`	latest at install time
`langchain`	$\geq$ 0.1.0
`langgraph`	$\geq$ 0.1.0

The environment variable `KMP_DUPLICATE_LIB_OK=TRUE` is set at runtime to resolve an OpenMP conflict between PyTorch and other native libraries.

11.2 Model versions/APIs used

Embedding model. `jinaai/jina-clip-v2` is loaded via `sentence-transformers` and `open_clip_torch`.

LLM / VLM.. `qwen3-vl:8b-instruct` is served through the remote Ollama instance and fulfils three roles across the pipeline:

- **Text generation / answer synthesis**: standard RAG answer generation (`BaselineConfig.LLM_MODEL`).
- **Visual classification / vision-language answering**: invoked before retrieval and when page images are retrieved (`AdvancedConfig.VLM_MODEL`).
- **Agent decisions**: query rewriting, document - confidence grading, and prompt/generator strategy selection in System 3 (`AgenticConfig.AGENT_LLM_MODEL`).

A model-specific workaround is active: `VLM_USE_RAW_CHATML = True` sends raw ChatML with a `/no_think` prefix instead of using the standard API `think=False` flag, because the Ollama-hosted `qwen3-vl:8b` ignores that parameter and otherwise produces verbose chain-of-thought output.

Generation parameters are fixed across all systems: temperature 0.0, top-p 0.1, max tokens 1024, with up to 4 retries on failure and a 30-second inter-retry delay.

Sparse retrieval. BM25 is implemented via the `rank-bm25` library and combined with dense retrieval in a hybrid retrieval scheme.

PDF parsing. Text is extracted with `pdfplumber` and `docling`. Images sent to the VLM are resized to a maximum of 1024×1024 pixels at JPEG quality 85 using `Pillow` to prevent GPU out-of-memory errors.

11.3 Hardware usage

Inference (LLM/VLM).. All model inference runs on the remote university Ollama server. The local machine makes no GPU calls for generation. Because requests go to a shared server, latency varies with server load. Generation workers are intentionally limited to one (`GENERATION_WORKERS = 1`) to avoid concurrent calls on the shared GPU, which could cause out-of-memory crashes. Retrieval workers are likewise set to one (`RETRIEVAL_WORKERS = 1`) for the multimodal and agentic pipelines.

Local machine. The local machine handles embedding computation (CPU or local GPU if available), Qdrant in-memory vector storage persisted to `src/local_qdrant/`, and pipeline orchestration. Embeddings are batched at 64 items (`EMBEDDING_BATCH_SIZE = 64`).

Query cache. A SQLite cache at `src/cache/query_cache.db` stores LLM responses keyed by (query, model, technique) to avoid redundant API calls during repeated evaluations.

11.4 Run commands

Detailed run commands for all three systems, including index setup, evaluation flags, and experiment configurations, are provided below. The same information is also available in the project's `README.md` file in the GitHub repository for a more structured reference.

11.5 Setup and Installation

Environment Requirements. Python 3.11 is required. Two setup options are provided:

(1) **Using Conda (Recommended):**
Conda creates an isolated Python environment so dependencies don't conflict with your system or other projects.

```
# Create and activate conda environment
conda create --name dat560project python=3.11
conda activate dat560project

# Install dependencies
pip install -r requirements.txt
```

(2) **Using Python's built-in venv:**
If you don't have Conda installed, use Python's built-in venv module instead.

```
# Create virtual environment with Python 3.11
python3.11 -m venv dat560project
```

macOS/Linux:

```
source dat560project/bin/activate
```

Windows:

```
dat560project\Scripts\activate
```

Then install dependencies:

```
pip install -r requirements.txt
```

To deactivate when done:

```
deactivate
```

GPU Support (Optional, Windows/Linux Only):
Only follow this step if you have an NVIDIA GPU and want faster computation. macOS users should skip this.

```
# Uninstall default CPU-only PyTorch
pip uninstall torch torchvision torchaudio -y

# Install CUDA-enabled PyTorch
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

Environment variables.
Copy the example file and fill in your credentials:

```
cp src/.env.example src/.env
```

```
OLLAMA_API_KEY=your_key_here      # Provided by the university
```

11.6 Data

Download the MMDocIR subset and place it under `src/data/`:

```
src/data/
|-- train/
|   |-- pdfs_train/
|   |-- page_images_train/
|   |-- images_train/
|   `-- train.jsonl
`-- test/
    |-- pdfs_test/
    |-- page_images_test/
    |-- images_test/
    `-- test.jsonl
```

11.7 Running the Pipeline

11.7.1 *Step 1 — Preprocessing (run once).* :

Extracts text and images from PDFs, applies all chunking strategies, and builds Qdrant indexes.

```
cd src

# Full run: PDF extraction + chunking + build all multimodal indexes
python -m pipelines.preprocessing_pipeline
```

What gets created. :

By default, the script rebuilds all Qdrant collections (one per chunking strategy) but **skips PDF extraction** (assumes `all_documents.json` already exists).

To control which stages run, edit `src/pipelines/preprocessing_pipeline.py` lines 79–80:

```
result = preprocessing_pipeline(
    force_chunking=False,   # Set to True to re-extract PDFs + apply chunking
    force_indexing=True,    # Set to True to rebuild all index collections
)
```

This creates 5 Qdrant collections in your local database:

- `advanced_fixed_size`
- `advanced_sliding_window`
- `advanced_semantic`
- `advanced_hierarchical`
- `advanced_enhanced_hierarchical`

Each contains the same documents but chunked differently. You select which one to use when running System 1 or System 2 (see Configuration section below).

Note: Its possible to do this step in Google Colab, read the `README.md` for instructions on this.

11.7.2 *Step 2 — Run Systems 1 & 2.*

System 1: Baseline RAG (Text-only). :

To run **System 1 only**, uncomment the code block below in `main.py` (lines 320–329) and **comment out everything else** in the `else:` statement:

```
# TO RUN BASELINE (System 1):
config = BaselineConfig()
pure_text_data = load_train_data(config.TEST_JSONL)
run_single_experiment(
    experiment_name="Baseline RAG",
    config=config,
    pipeline_class=BaselineRAGPipeline,
    test_data=pure_text_data[:config.EVAL_SUBSET_SIZE],
    force_rebuild=False,
    run_single_query=True
)
```

Then run:

```
cd src
python main.py
```

This uses the baseline configuration from `src/config/config.py` (fixed_size chunking, standard query technique, standard prompting).

System 2: Advanced mRAG (Multimodal). :

Option A: Run with Configuration
Uncomment this block in `main.py` (lines 339–353) and comment out everything else:

```
# TO RUN:
config = AdvancedConfig(
    CHUNKING_STRATEGY="enhanced_hierarchical",  # or "semantic"
    QUERY_TECHNIQUE="rag_fusion",               # or "hyde", "multi_query", etc.
    PROMPTING_STRATEGY="cot",                   # Chain of Thought
    USE_MULTIMODAL=True                         # If its to use images or not.
)
pure_text_data = load_train_data(config.TEST_JSONL)
run_single_experiment(
    experiment_name="Advanced RAG (Best Config)",
    config=config,
    pipeline_class=AdvancedRAGPipeline,
    test_data=pure_text_data,
    force_rebuild=False,
    run_single_query=True
)
```

Then run:

```
cd src
python main.py
```

This runs **one experiment** with your chosen parameters. You can modify `CHUNKING_STRATEGY`, `QUERY_TECHNIQUE`, and `PROMPTING_STRATEGY` in this block to test different configurations.

Option B: Run ALL Ablation Experiments
Uncomment this block in `main.py` (line 333) and comment out everything else:

```
# TO RUN ALL ABLATION TESTS:
run_experiments(eval_subset_size=1000)
```

Then run:

```
cd src
python main.py
```

This systematically runs:

- 1 Baseline RAG
- 4 Chunking strategy ablations (sliding_window, semantic, hierarchical, enhanced_hierarchical)
- 7 Query technique ablations (multi_query, rag_fusion, step_back, hyde, query_decomposition, query_rewriting, query_expansion)
- 3 Prompting strategy ablations (few_shot, role, cot)
- 1 Multimodal variant

Option C: Run Incremental Optimization (Greedy Search)
Uncomment this block in `main.py` (line 336) and comment out everything else:

```
# TO RUN INCREMENTAL ADDITION (Step-by-step optimization):
run_incremental_addition(eval_subset_size=1000, target_metric='token_f1')
```

Then run:

```
cd src
python main.py
```

This uses **greedy optimization**: sequentially tests each component (chunking, query technique, prompting strategy, multimodal), carrying forward only the best-performing option at each step. Much faster than all ablations, but less comprehensive.

Run Single Experiments from CLI. :

Instead of editing main.py, you can run single experiments directly from the terminal using the `-run-experiments` flag:

```
# EXAMPLES:
python main.py --run-experiments --technique multi_query --prompting-strategy cot

python main.py --run-experiments --technique rag_fusion --eval-subset 30

python main.py --run-incremental --incremental-metric exact_match

python main.py --run-experiments --technique hyde --force-rebuild
```

Important: When using CLI flags, you must include `-run-experiments` or `-run-incremental` for them to take effect. Without these flags, the hardcoded default config runs instead. **CLI Flags Reference:**

Flag	Type	Default	Description
`-run-experiments`	Flag	—	Run a single experiment with specified technique and strategy
`-run-incremental`	Flag	—	Run greedy optimization selecting best at each step
`-technique`	String	standard	Query technique (only used with –run-experiments)
`-prompting-strategy`	String	standard	Prompting strategy (only used with –run-experiments)
`-eval-subset`	Integer	20	Number of test queries to evaluate
`-incremental-metric`	String	token_f1	Target metric to optimize (only used with –run-incremental)
`-force-rebuild`	Flag	—	Force rebuild of Qdrant index

Results are saved to:

- Single experiments: `src/results/pipeline_results.csv`
- All ablations: `src/results/results_experiments.csv`
- Incremental optimization: `src/results/pipeline_results.csv` (timestamped rows)

Text-Only Mode (Disable Multimodal). :

For Option A (code block): Change `USE_MULTIMODAL=False` directly in the code block:

```
config = AdvancedConfig(
    CHUNKING_STRATEGY="enhanced_hierarchical",
    QUERY_TECHNIQUE="rag_fusion",
    PROMPTING_STRATEGY="cot",
    USE_MULTIMODAL=False  # Change to False here
)
```

For Options B, C, and CLI modes: Edit `src/config/config.py` and change `USE_MULTIMODAL` in the `AdvancedConfig` class:

```
USE_MULTIMODAL: bool = False
```

Then run any of these commands (EXAMPLES):

```
python main.py --run-experiments --technique rag_fusion

python main.py --run-incremental --incremental-metric token_f1

python main.py  # (if you uncommented Option B or C)
```

This completely disables multimodal retrieval—no images are retrieved or processed, only text.

11.7.3 ***Step 3 — Run System 3 (Agentic).*** :

The below code block presents how to run the System 3 evaluation (either with a single query/question, or on the whole evaluation (testing) set).

```
cd src

# Testing with a single query
python main_agentic.py --test-query "How many students of NTU would recommend studying at NTU?"

# Running the full evaluation (on test set)
python main_agentic.py --eval --eval-size 150 --output results_agentic.json
```

The agentic system prints each agent's decision (query technique chosen, grader confidence, prompting strategy), per query. You can also specify a JSON output file to store information and output, if further inspection is necessary.
Note: Running the above-mentioned System 3 commands assumes that the index is already built.

11.8 Configuration

All settings live in `src/config/config.py`. Three config classes are available: `BaselineConfig` (System 1), `AdvancedConfig` (System 2), `AgenticConfig` (System 3).

11.8.1 ***Selecting a Chunking Strategy & Collection.*** :

`CHUNKING_STRATEGY` and `VECTOR_DB_COLLECTION` are **independent settings** that must match:

CHUNKING_STRATEGY	Should use Collection
fixed_size	advanced_fixed_size
sliding_window	advanced_sliding_window
semantic	advanced_semantic
hierarchical	advanced_hierarchical
enhanced_hierarchical	advanced_enhanced_hierarchical

When using `-run-experiments` from CLI:. The collection is set automatically based on `CHUNKING_STRATEGY`:

```
python main.py --run-experiments --technique rag_fusion \
  --chunking-strategy semantic
# Automatically sets: VECTOR_DB_COLLECTION = "advanced_semantic"
```

When running System 1 or System 2 directly: (without `-run-experiments`): You must manually set both in `src/config/config.py` to match:

```
# In AdvancedConfig (around line 135):
CHUNKING_STRATEGY: str = "semantic"
VECTOR_DB_COLLECTION: str = "advanced_semantic"  # Must match!
```

Important: The collection must already exist from preprocessing (see Step 1 above). Alternatively, you can rebuild it with the `-force-rebuild` flag:

```
python main.py --run-experiments --technique rag_fusion --force-rebuild
```

Configuration Parameters:

Parameter	Default	Options/Notes
`CHUNKING_STRATEGY`	`fixed_size`	`fixed_size`, `sliding_window`, `semantic`, `hierarchical`, `enhanced_hierarchical`
`QUERY_TECHNIQUE`	`standard`	`standard`, `multi_query`, `rag_fusion`, `step_back`, `hyde`, `query_rewriting`, `query_expansion`, `query_de`
`PROMPTING_STRATEGY`	`standard`	`standard`, `cot`, `few_shot`, `role`, `ensemble`
`USE_MULTIMODAL`	`True`	Enable multimodal retrieval (page images, figures, evidence crops)
`TOP_K`	5	Number of retrieved chunks per query
`EVAL_SUBSET_SIZE`	20	Number of test questions to evaluate
`VECTOR_DB_MODE`	`local`	`local` (disk), `memory` (RAM), `docker` (remote Qdrant)
`LLM_MODEL`	`qwen3-vl:8b-instruct`	Any model available via Ollama
`LLM_TEMPERATURE`	0.0	Generation temperature (reduce hallucination)
`AGENT_MAX_RETRIES`	1	Specify number of retry-attempts available
`GRADER_CONFIDENCE_THRESHOLD`	0.51	Specify retry confidence threshold

11.9 Evaluation Metrics

Retrieval:

Metric	Description
Precision@k	Fraction of retrieved docs in top-k that are relevant
Recall@k	Fraction of all relevant docs that appear in top-k
Page Recall@k	Binary hit: 1.0 if any retrieved chunk in top-k matches relevant PDF and overlaps relevant pages, else 0.0
MAP	Mean Average Precision across all queries; average precision at each position where a relevant doc is found
MRR	Mean Reciprocal Rank of the first relevant result (1 / rank of first hit)
NDCG@k	Normalized Discounted Cumulative Gain; ranking quality with position weighting (DCG@k / IDCG@k)

Generation:

Metric	Description
Exact Match	Normalized string equality with ground truth (after lowercasing, punctuation removal, etc.)
Contains Match	Binary: 1.0 if ground truth value appears anywhere in the prediction, else 0.0
Token F1	SQuAD-style token-level F1 score; measures token overlap frequency between prediction and answer
BLEU	Bilingual Evaluation Understudy; n-gram overlap score (BLEU-4: up to 4-grams) with brevity penalty
ROUGE-1	Recall-Oriented Understudy for Gisting Evaluation; unigram (1-word) overlap F-measure
ROUGE-2	ROUGE for bigrams (2-word sequences) F-measure
ROUGE-L	ROUGE for longest common subsequence F-measure
Semantic (Textual) Similarity	Embedding-based cosine similarity between prediction and answer embeddings

11.10 Reproducibility

Mechanism	Detail
Prompt templates	Versioned in `src/generation/prompts/`
Preprocessing artifacts	Chunk files cached in `src/data/preprocessed/` and reused
Index reuse	Qdrant collections reused unless `--force-rebuild` is passed
Deterministic generation	`LLM_TEMPERATURE = 0.0`, `LLM_TOP_P = 0.1`
Timing logs	Preprocessing and experiment runtimes saved to `src/results/`

For complete and up-to-date instructions, refer to `README.md` in the project repository.